How Many Bits Does a Training Example Cost? Per-Example Capacity versus Fidelity via Linearized Laplace Unlearning

Matthew J. O'Meara[1,2]

[1]Gilbert S. Omenn Department of Computational Medicine and Bioinformatics, University of Michigan, Ann Arbor, MI, USA

[2]Department of Medicinal Chemistry, University of Michigan, Ann Arbor, MI, USA

maom@umich.edu

Abstract

The compression/fidelity trade-off is usually stated for a whole model: parameters are a code whose length is traded against the code length of the data given those parameters. We ask the per-example version of the question: how much of a trained network's capacity is spent on each training example (or set of examples), and how well is that example fit? We define capacity by *unlearning*: the change in a linearized Laplace posterior over the weights when an example is removed, measured as (i) the effective number of parameters it determines, (ii) the drop in posterior differential entropy (a label-agnostic *volume*), and (iii) the Kullback–Leibler divergence between the posteriors with and without it (the label-dependent *information* actually stored). All three reduce to $C \times C$ output-space matrices, so every example is scored after a single factorization, and arbitrary sets are scored jointly. Under exact Bayesian updating, information capacity equals the example's generalization gap (training minus held-out log-likelihood); volume equals the mutual information between weights and label for Gaussian likelihoods; and volume is submodular, which yields a redundancy ratio for sets. In a closed-form linear model all identities hold to 10^{-13} bits. On a two-moons MLP with 10% flipped labels, sweeping the prior precision moves flipped examples from *ignored* (low fidelity, $\sim$0.08 bits each) to *memorized* (high fidelity, $\sim$1.6 bits each), while the Laplace evidence favors the most compressed model. On a 4,130-parameter vision transformer trained on 8×8 digits with 5% flipped labels, flipped examples cost a median of 3.5 bits versus 0.05 bits for clean ones, 14 of the 16 most expensive images are flips, and capacity rank-correlates with memorization measured by actual retraining (Spearman 0.84–0.88). The one-step held-out prediction, however, overestimates the effect of removing heavily memorized examples by up to several-fold, which we document as the main limitation. Computing the posterior in output space through the empirical neural tangent kernel removes the dependence on the number of parameters, which lets us analyze 18 molecular property-prediction networks of up to 4.2 million parameters: capacity ranks the compounds those networks memorize, as measured by 864 retrainings (Spearman 0.38–0.73), and the compounds it flags are label conflicts with structural alerts, ionizable compounds whose $\log D$ the representations cannot express, inorganic salts, and groups of compounds that a fingerprint maps to one vector but whose labels span six log units.

Code: https://github.com/maomlab/capfid.

1 Introduction

Minimum description length (MDL) and the information bottleneck frame learning as a trade-off between how much information a model stores and how faithfully it reproduces data (Rissanen, 1978; Hinton and van Camp, 1993; Tishby et al., 2000; Grünwald, 2007). A model with a short parameter code that explains the data well generalizes; a model that stores many bits to fit a few examples memorizes. These statements are global. In practice one often wants the *per-example* decomposition: which examples consume capacity, how much, and what the model gets in return in fidelity. This matters for auditing memorization (Zhang et al., 2017; Feldman, 2020; Feldman and Zhang, 2020), diagnosing label noise, identifying long-tail subpopulations, and valuing data (Ghorbani and Zou, 2019; Pruthi et al., 2020; Covert et al., 2024), a question economists study as the value of information (Raiffa and Schlaifer, 1961; Abadie et al., 2023).

Two families of tools approximate pieces of this picture. Influence functions estimate how parameters or predictions change when an example is removed (Cook, 1977; Koh and Liang, 2017), and memorization scores compare held-in and held-out likelihood, typically by retraining many models (Feldman and Zhang, 2020). Both ask what changes when an example is withheld, but they answer in incommensurable units, a parameter shift or a gap in log-likelihood, neither of which says what the example cost the model. Information-theoretic work supplies a common currency for that comparison: the information an example leaves in the weights, defined as the divergence between weight distributions trained with and without it (Achille et al., 2019; Harutyunyan et al., 2021), and measured in bits like the description lengths above. We build on this line and make three things explicit: a family of capacity measures with distinct meanings, identities that tie them to fidelity and to classical quantities, and a two-dimensional *capacity–fidelity* diagnostic that can be computed for every example of a small network in seconds.

Contributions.

- We define three nested per-example (and per-set) capacity measures by Laplace unlearning: effective degrees of freedom, volume (entropy drop), and information (KL), and show they reduce exactly to $C \times C$ output-space quantities,

where C is the number of outputs of the network (Proposition 1).

- We prove or collect identities that give them meaning: information equals the generalization gap under exact Bayesian updating (Proposition 2); volume equals the weight–label mutual information for Gaussian likelihoods and the leverage formula $-\frac{1}{2}\log(1-h_{ii})$ in linear regression; degrees of freedom is exactly additive, so per-example values sum to MacKay's effective number of parameters γ (Remark 1); and volume is submodular, giving a redundancy ratio ≥ 1 for sets and an upper bound on the fraction of model information attributable to single examples (Proposition 3).
- We connect the totals to a Laplace two-part MDL code (Section 3.3) and demonstrate the diagnostic on an exactly solvable model, an MLP under a prior-precision sweep, and a small vision transformer, validating against warm-started retraining and documenting where the approximation fails.
- We compute the same posterior in output space through the empirical neural tangent kernel, at a cost independent of the number of parameters, and refit its mean for networks that are not at a stationary point, such as those trained with dropout or early stopping (Section 3.5). This makes the analysis practical for networks with millions of parameters and a few thousand training examples, which we use for a case study of molecular property prediction (Section 5).

2 Background and related work

Laplace approximations. The Laplace approximation fits a Gaussian at a mode using the curvature of the negative log posterior (MacKay, 1992). For neural networks the generalized Gauss–Newton (GGN) curvature is positive semidefinite and corresponds to a linearization of the network in its parameters (Ritter et al., 2018; Immer et al., 2021; Daxberger et al., 2021). The Laplace marginal likelihood penalizes models by an Occam factor that MacKay related to the effective number of well-determined parameters, γ.

Leverage and influence. In linear regression the diagonal h_{ii} of the hat matrix measures how much an observation determines its own fit, and $\sum_i h_{ii}$ is the model's degrees of freedom (Hoaglin and Welsch, 1978; Cook, 1977). Influence functions generalize the parameter shift to differentiable models (Koh and Liang, 2017), but are known to be fragile for deep networks, especially for highly influential points (Basu et al., 2021; Bae et al., 2022).

Information in the weights and generalization. Achille et al. (2019) define the information a dataset stores in the weights as a KL to a prior, and Harutyunyan et al. (2021) introduced *smooth unique information*: a KL between weight distributions trained with and without an example, computed with a linearized network. Information-theoretic generalization bounds control the gap between train and test loss by the mutual information between weights and individual examples (Xu and Raginsky, 2017; Bu et al., 2020). Our Proposition 2 is the pointwise, exact-posterior version of this connection. MDL views of deep learning (Hinton and van Camp, 1993; Blier and Ollivier, 2018) motivate reading the Laplace evidence as a two-part code.

Value of information and the economics of data. Statistical decision theory values data by what it does for decisions: the expected value of sample information is the expected improvement in the payoff of the best action after observing data, net of the cost of collecting it (Blackwell, 1951; Raiffa and Schlaifer, 1961; Howard, 1966). Abadie et al. (2023) turned this into an empirical-Bayes estimator of the value of evidence-based decision making. A decision maker with prior $\tau \sim \mathcal{N}(\mu, \omega^2)$ on a policy's payoff can buy a signal $\hat{\tau} \sim \mathcal{N}(\tau, \sigma^2)$ and launch only if the posterior mean is positive; the value of evidence is the resulting gain in expected payoff minus its cost, estimated from many past experiments. They show how this value changes with statistical precision and that launch rules based on statistical significance can leave it unrealized. Two connections to our measures are exact. First, when the utility is the logarithmic score of a reported belief about the parameters, the expected value of an experiment is its expected KL divergence from prior to posterior, that is, the mutual information between parameters and data (Lindley, 1956; Bernardo, 1979). By Corollary 1, our volume $\mathcal{V}(S)$ is exactly this quantity for the labels of S given the rest of the data (for Gaussian likelihoods): it is the *ex ante* value of those labels to such a decision maker, and $\mathcal{I}(S)$ is their *ex post* value once observed. Second, in their Gaussian model the value of evidence depends on precision only through the shrinkage weight $h = \omega^2/(\omega^2+\sigma^2)$, which is the single-observation leverage K of Proposition 1; the corresponding volume is $-\frac{1}{2}\log(1-h) = \frac{1}{2}\log(1+\omega^2/\sigma^2)$. The frameworks differ in what is valued. Theirs prices evidence by a downstream decision and its cost; ours prices training examples in model capacity and fit, with no decision layer. Selecting α by maximizing the Laplace evidence (Section 3.3), which we report but did not use to set α, would be the same empirical-Bayes step they use to estimate the prior from a population of studies.

Data valuation, scaling laws, and large language models. Data valuation in machine learning assigns each example a share of held-out performance, for example through Shapley values (Ghorbani and Zou, 2019) or gradient tracing (Pruthi et al., 2020). Covert et al. (2024) found that an example's marginal contribution to performance shrinks log-linearly with the size of the dataset it joins, with example-specific exponents: some examples matter most in small datasets and others remain useful in large ones. Our capacity is an information-theoretic counterpart of their marginal contribution, evaluated at the full dataset (a subset of size $N-1$) and measured as information stored rather than held-out loss. With curvatures held at θ^*, Proposition 3 implies that an example's marginal volume can only shrink as the dataset it joins grows, the diminishing returns their scaling laws quantify, and the Laplace approximation gives

this marginal volume in closed form for any subset, so information-scaling curves could be traced without retraining. At the largest scale, influence functions with eigenvalue-corrected Kronecker-factored curvature have been applied to language models with up to 52 billion parameters to trace behaviors back to pretraining documents (Grosse et al., 2023). The same curvature approximations are the natural route to scaling our capacity measures (Section 3.4).

Local surrogates and feature attribution. A large explainability literature asks what a prediction depends on rather than what a training example costs. LIME explains one prediction by fitting an interpretable surrogate, typically a sparse linear model, to the black box in a neighborhood of the input, on the premise that a model too complicated to describe globally is simple enough to describe locally (Ribeiro et al., 2016). We adopt the same premise in weight space rather than input space: Equation (2) replaces the network by a surrogate that is linear in θ near θ^*, and every quantity we report is read off that surrogate. The two differ in what the surrogate is local to and what it yields. LIME's is local to one input, fit by sampling perturbations of it, and yields a feature attribution; ours is local to one parameter vector, obtained from the curvature of the training objective, and yields a posterior, which is what lets us delete data from it in closed form and report the result in bits. SHAP attributes a prediction to features by the Shapley value of a cooperative game among them, whose payoff is the model's expected output, or its performance in loss-based variants, when a coalition of features is supplied and the rest are withheld (Lundberg and Lee, 2017); the same game played over training examples rather than features is Data Shapley (Ghorbani and Zou, 2019). In contrast to SHAP, we hold the features fixed and withhold data; in contrast to Data Shapley, we report two quantities in place of a single payoff: the capacity a set of examples occupies and the fidelity with which it is fit, so that an example a model fits for free and one it fits by spending capacity are distinguished rather than summarized by one number. We also evaluate a single coalition, the full training set, instead of averaging over all of them. That is why a score costs one posterior and no refitting, and also why it is conservative: by Proposition 3 leave-one-out marginals undercount the total, and the Shapley-style average over removal orders that would distribute $\mathscr{V}_{\text{tot}}$ exactly remains an extension.

3 Method

3.1 Setup

Let $\mathscr{D} = \{(x_i, y_i)\}_{i=1}^N$ be a dataset of N examples, let $f(x;\theta) \in \mathbb{R}^C$ be a network with P parameters $\theta \in \mathbb{R}^P$ that maps an input x to C outputs, one per class for a softmax classifier and $C = 1$ for scalar regression or a single logit, and let $\ell(y, f) = -\log p(y \mid f)$ be the loss, the negative log-likelihood of the label y given those outputs. We train a MAP estimate θ^* of

$$L(\theta) = \textstyle\sum_{i=1}^N \ell\big(y_i, f(x_i;\theta)\big) + \frac{\alpha}{2}\|\theta\|^2, \tag{1}$$

i.e., a summed loss with a Gaussian prior $\mathcal{N}(0, \alpha^{-1}I)$. At θ^* define the Jacobian $J_i = \partial f(x_i)/\partial\theta \in \mathbb{R}^{C\times P}$, the output-space residual $r_i = \nabla_f \ell \in \mathbb{R}^C$, and curvature $\Lambda_i = \nabla_f^2 \ell \in \mathbb{R}^{C\times C}$ (for softmax, $r_i = p_i - e_{y_i}$ and $\Lambda_i = \operatorname{diag}(p_i) - p_i p_i^\top$, where $p_i = \operatorname{softmax}(f(x_i)) \in \mathbb{R}^C$ holds the predicted class probabilities and $e_{y_i} \in \mathbb{R}^C$ is the one-hot vector of the observed class y_i). The linearized Laplace posterior is

$$q_{\mathscr{D}} = \mathcal{N}(\theta^*, \Sigma), \qquad \Sigma^{-1} = A = \textstyle\sum_i J_i^\top \Lambda_i J_i + \alpha I. \tag{2}$$

Unlearning a set. For $S \subseteq [N]$ with $m = |S|$, stack $J_S \in \mathbb{R}^{mC\times P}$, $r_S \in \mathbb{R}^{mC}$, and $\Lambda_S = \operatorname{blkdiag}(\Lambda_i)_{i\in S}$. Removing S subtracts its curvature and, because $\nabla L(\theta^*) = 0$, one Newton step on $L - \sum_{i\in S}\ell_i$ gives

$$A_{-S} = A - J_S^\top \Lambda_S J_S, \qquad \theta^*_{-S} = \theta^* + A_{-S}^{-1} J_S^\top r_S, \tag{3}$$

and $q_{\mathscr{D}\setminus S} = \mathcal{N}(\theta^*_{-S}, A_{-S}^{-1})$. This is exact for linear-Gaussian models.

3.2 Capacity and fidelity

Definition 1 (Capacity)**.** For a set S,

$$\begin{aligned} d(S) &= \operatorname{tr}\big(\Sigma J_S^\top \Lambda_S J_S\big) && \text{(degrees of freedom)} \\ \mathscr{V}(S) &= h(q_{\mathscr{D}\setminus S}) - h(q_{\mathscr{D}}) && \text{(volume)} \\ \mathscr{I}(S) &= \mathrm{KL}\big(q_{\mathscr{D}} \,\|\, q_{\mathscr{D}\setminus S}\big) && \text{(information)} \end{aligned}$$

with h the differential entropy. We report $\mathscr{V}$ and $\mathscr{I}$ in bits.

$\mathscr{V}$ is the log-ratio of weight-space volumes the examples pin down; it depends on the inputs and on the rest of the data, but not on the labels y_S except through Λ_S. $\mathscr{I}$ adds a mean-shift term and measures what is actually stored about the labeled examples. Unlike an absolute differential entropy, $\mathscr{V}$ is invariant to affine reparameterizations of θ (and, within the Laplace approximation, to any smooth reparameterization, which is linearized at θ^* so that the Jacobian terms cancel); $\mathscr{I}$ is invariant to any bijection.

Remark 1 (The three totals, and γ)**.** Unlearning all of $\mathscr{D}$ returns the prior ($A_{-\mathscr{D}} = \alpha I$, and $\theta^*_{-\mathscr{D}} = 0$ because $\nabla L(\theta^*) = 0$), so evaluating Definition 1 at $S = \mathscr{D}$ gives the three model-level totals:

$$d(\mathscr{D}) = \gamma, \quad \mathscr{V}(\mathscr{D}) = \mathscr{V}_{\text{tot}}, \quad \mathscr{I}(\mathscr{D}) = \mathrm{KL}(q_{\mathscr{D}} \| \text{prior}), \tag{4}$$

where $\gamma = \operatorname{tr}(\Sigma\,\mathrm{GGN}) = P - \alpha\operatorname{tr}\Sigma$ is MacKay's effective number of parameters (MacKay, 1992) and $\mathscr{V}_{\text{tot}} = \frac{1}{2}\log\det(I + \mathrm{GGN}/\alpha)$ is the bits by which the data shrink the weight space. Every γ in this paper is this total. Because it is a trace, d is the only one of the three that is exactly additive,

$$d(S) = \textstyle\sum_{i\in S} d(\{i\}), \qquad \sum_{i=1}^N d(\{i\}) = \gamma, \tag{5}$$

so $d(\{i\})$ is example i's exact share of the model's effective parameters. $\mathscr{V}$ is superadditive instead (Proposition 3), and $\mathscr{I}$ is not additive either, which is why sets need their own scores.

Definition 2 (Fidelity)**.** The fidelity of example i is $F_i = \log_2 p(y_i \mid x_i, \theta^*)$ and its held-out fidelity is $F_i^- = \log_2 p(y_i \mid x_i, \theta^*_{-i})$, using the linearized held-out logits of Proposition 1. Posterior-averaged versions are $\bar{F}_i = \mathbb{E}_{q_\mathscr{D}} \log_2 p(y_i \mid \theta)$ and $\tilde{F}_i^- = \log_2 \mathbb{E}_{q_{\mathscr{D}_{\setminus i}}} p(y_i \mid \theta)$.

Proposition 1 (Output-space reduction)**.** *Let* $S_S = J_S \Sigma J_S^\top$, $K_S = \Lambda_S^{1/2} S_S \Lambda_S^{1/2}$ *and* $S_{-S} = J_S A_{-S}^{-1} J_S^\top = S_S + S_S \Lambda_S^{1/2} (I - K_S)^{-1} \Lambda_S^{1/2} S_S$. *Then, in nats,*

$$d(S) = \operatorname{tr} K_S, \qquad \mathscr{V}(S) = -\tfrac{1}{2} \log\det(I - K_S),$$
$$\mathscr{I}(S) = \tfrac{1}{2} \Big[r_S^\top S_{-S} r_S - \operatorname{tr} K_S - \log\det(I - K_S) \Big],$$

and the linearized held-out logits are $f(x_S) + S_{-S} r_S$.

All quantities involve only $mC \times mC$ matrices; for single examples they are $C \times C$. The proof (Appendix B) uses Sylvester's determinant identity and the Woodbury identity. K_S is a block of the posterior "hat" matrix: it is the leverage of S under the Laplace posterior. Since $A_{-S} \succeq \alpha I$, the eigenvalues of K_S lie in $[0, 1)$.

Proposition 2 (Information is the generalization gap)**.** *If* $q_\mathscr{D} \propto q_{\mathscr{D}_{\setminus S}} p(y_S \mid \theta)$ *exactly, then*

$$\mathscr{I}(S) = \mathbb{E}_{q_\mathscr{D}} \big[\log p(y_S \mid \theta) \big] - \log \mathbb{E}_{q_{\mathscr{D}_{\setminus S}}} \big[p(y_S \mid \theta) \big].$$

That is, the bits the posterior stores about S equal the difference between its posterior-averaged training fit and its leave-S-out predictive fit. The condition holds for linear-Gaussian models, where Equation (3) is exact. On a plot of $\bar{F}_i$ against $\mathscr{I}(\{i\})$, lines of slope one are then iso-lines of held-out likelihood.

Corollary 1 (Volume as mutual information)**.** *For a Gaussian likelihood,* Σ *does not depend on* y*, and* $\mathbb{E}_{y_S \sim p(\cdot \mid \mathscr{D}_{\setminus S})} \mathscr{I}(S) = \mathscr{V}(S) = I(\theta; y_S \mid x_S, \mathscr{D}_{\setminus S})$. *For linear regression with a Gaussian prior,* $\mathscr{V}(\{i\}) = -\tfrac{1}{2} \log(1 - h_{ii})$ *where* h_{ii} *is the leverage of the Bayesian ridge hat matrix.*

So $\mathscr{V}$ is the *capacity allotted* to an input location (the expected information about any plausible label), while $\mathscr{I}$ is the information spent on the observed label. Expanding in the eigenvalues κ of K_S: $\mathscr{V} \approx \frac{1}{2} \sum \kappa$, whereas the covariance part of $\mathscr{I}$ is $\frac{1}{2} \sum [-\kappa - \log(1 - \kappa)] \approx \frac{1}{4} \sum \kappa^2$. For well-fit examples (small r), $\mathscr{I} \approx \mathscr{V}^2$ in nats: they occupy volume but store little. Examples with surprising labels add the mean-shift term and can have $\mathscr{I} > \mathscr{V}$.

Proposition 3 (Redundancy)**.** *For every* S, $\mathscr{V}(S) \geq \sum_{i \in S} \mathscr{V}(\{i\})$, *and the total model information* $\mathscr{V}_{\text{tot}} = h(\text{prior}) - h(q_\mathscr{D}) = \frac{1}{2} \log\det(I + \text{GGN}/\alpha)$ *satisfies* $\sum_{i=1}^N \mathscr{V}(\{i\}) \leq \mathscr{V}_{\text{tot}}$.

This follows from submodularity of $T \mapsto \log\det(\alpha I + \sum_{i \in T} J_i^\top \Lambda_i J_i)$ (Krause et al., 2008). We report the *redundancy ratio* $\rho(S) = \mathscr{V}(S) / \sum_{i \in S} \mathscr{V}(\{i\}) \geq 1$, which is large when members of S encode the same information, and the *unique fraction* $u = \sum_i \mathscr{V}(\{i\}) / \mathscr{V}_{\text{tot}} \leq 1$.

Algorithm 1 Per-example capacity and fidelity

$J \leftarrow$ per-example Jacobians ($N \times C \times P$; `vmap(jacrev)`)
$A \leftarrow \alpha I + \sum_i J_i^\top \Lambda_i J_i$; $L \leftarrow \text{chol}(A)$
for each batch B **do**
  $V \leftarrow L^{-1} J_B^\top$; $S_i \leftarrow V_i^\top V_i$ for $i \in B$
end for
for each i (batched $C \times C$ algebra) **do**
  K_i, S_{-i}, d, $\mathscr{V}$, $\mathscr{I}$ via Proposition 1
  F_i from $f(x_i)$; F_i^- from $f(x_i) + S_{-i} r_i$
end for

3.3 Totals as a two-part code

The Laplace log evidence decomposes as $-\log p(\mathscr{D}) \approx L_{\text{data}} + L_{\text{model}}$ with $L_{\text{data}} = -\sum_i \log p(y_i \mid x_i, \theta^*)$ and $L_{\text{model}} = \frac{\alpha}{2} \|\theta^*\|^2 + \mathscr{V}_{\text{tot}}$. Moreover the total information of Remark 1 is $\mathscr{I}(\mathscr{D}) = \text{KL}(q_\mathscr{D} \| \text{prior}) = L_{\text{model}} - \gamma/2$. Varying α moves along a compression/fidelity curve, and the evidence selects a point on it.

3.4 Computation

Algorithm 1 costs $O(NCP^2 + P^3)$ time and $O(P^2 + NCP)$ memory for the exact GGN; set capacities need one extra triangular solve with $J_S^\top$. Expectations over q use Monte Carlo with common random numbers. The method is implemented in the open-source PyTorch package `capfid` (see Code availability). For larger models, only $J_i \Sigma J_i^\top$ is needed, so Kronecker-factored, last-layer, or low-rank curvature can be substituted (Martens and Grosse, 2015; Ritter et al., 2018; Daxberger et al., 2021).

3.5 Wide networks: computation in output space

Every quantity above needs only blocks of $J \Sigma J^\top$ and $\log\det(I + \text{GGN}/\alpha)$. With the empirical neural tangent kernel $G = J J^\top \in \mathbb{R}^{NC \times NC}$ (Jacot et al., 2018), $\Lambda = \text{blkdiag}(\Lambda_i)$, and $M = \alpha I + \Lambda^{1/2} G \Lambda^{1/2}$, the Woodbury and Sylvester identities give

$$J \Sigma J^\top = \alpha^{-1} \big[G - G \Lambda^{1/2} M^{-1} \Lambda^{1/2} G \big],$$
$$\log\det\big(I + \tfrac{\text{GGN}}{\alpha}\big) = \log\det \tfrac{M}{\alpha}. \tag{6}$$

One Cholesky factorization of M therefore yields the identical posterior in $O((NC)^3)$ time, independent of P: the exact Laplace posterior of a network with millions of parameters is available whenever the training set has a few thousand examples. For a linear layer $z = Wa + b$, $\partial f_c / \partial W = \delta_c a^\top$ with $\delta_c = \partial f_c / \partial z$, so for a multilayer perceptron $G = \sum_\ell (\Delta_\ell \Delta_\ell^\top) \odot (A_\ell A_\ell^\top + \mathbf{1})$ is assembled from layer inputs A_ℓ and output gradients Δ_ℓ in one forward and C backward passes, without forming J. The same algebra at a new input x gives the posterior output covariance $J_x \Sigma J_x^\top$ and a label-free *prospective volume* $\frac{1}{2} \log\det(I + \Lambda_x^{1/2} J_x \Sigma J_x^\top \Lambda_x^{1/2}) = h(q_\mathscr{D}) - h(q_{\mathscr{D} \cup \{x\}})$, the capacity x would occupy if it were added to the training set;

Table 1: Exact check in Bayesian linear regression: maximum absolute discrepancy over all 200 examples.

Quantity	max error (bits)
$\mathscr{I}$ (Proposition 1) vs. exact refit KL	1.6×10^{-13}
held-out log-lik vs. exact refit	2.8×10^{-14}
$\mathscr{I}$ vs. train − held-out (Proposition 2)	9.6×10^{-15}
$\mathscr{V}$ vs. $-\frac{1}{2}\log_2(1-h_{ii})$ (Corollary 1)	3.9×10^{-16}
$\gamma=\sum_i d(\{i\})$ vs. $\mathrm{tr}\,H$	9.091 vs. 9.091

for a Gaussian likelihood it is the mutual information between the unseen label and the weights.

Networks not trained to a mode. Equation (3) assumes $\nabla L(\theta^*)=0$. Networks trained with dropout or early stopping, or analyzed with a prior precision chosen after training, violate this, and a Newton step from θ^* then credits the residual gradient to whichever example is removed: high-leverage examples that the network did not fit receive enormous spurious memorization. We therefore center the posterior on the MAP of the linearized network $f(x;\theta^*)+J_x(\theta-\theta^*)$ instead. Viewing the linearized network as a Gaussian process with kernel G/α (Khan et al., 2019; Immer et al., 2021) and a constant prior mean m (we use the training-set average), this MAP is the posterior mode $\hat{f}$ at the training inputs, found by Newton's method (Rasmussen and Williams, 2006, Alg. 3.1). Residuals and curvatures are evaluated at $\hat{f}$, and Proposition 1 applies unchanged. For a Gaussian likelihood a single step is exact, unlearning becomes exact Gaussian-process leave-one-out, Proposition 2 holds exactly, and the Laplace evidence is the Gaussian-process marginal likelihood. Because the kernel G of a trained network has itself been fit to the training labels, maximizing that marginal likelihood over the noise σ lets the kernel absorb label noise as signal; we therefore choose α by the marginal likelihood but σ by held-out data (Section 5.1).

4 Experiments

The experiments in this section ran on a single CPU core in double-precision Laplace algebra; details are in Appendix C. The case study of Section 5 is larger and is described in Appendix D.

4.1 Exact check: Bayesian linear regression

Appendix A works this setting out from first principles; here we use it to check the implementation. We fit $N=200$ noisy samples of $\sin(2x)$ with $P=40$ random Fourier features ($\sigma=0.3$, $\alpha=1$; five targets shifted by $+2$ as outliers) and compared Propositions 1 and 2 and Corollary 1 with brute-force refits without each point (Table 1). All agree to numerical precision. Per-example degrees of freedom sum to $\mathrm{tr}\,H=9.091$ effective parameters, and outliers carry a mean of 1.34 bits versus 0.044 bits for inliers.

Table 2: Two-moons prior-precision sweep. $\gamma=d(\mathscr{D})$ and $\mathscr{V}_{\mathrm{tot}}=\mathscr{V}(\mathscr{D})$ are the totals of Remark 1; L values in bits; $\tilde{\mathscr{I}}$ is the median of $\mathscr{I}(\{i\})$ (bits); u is the unique fraction (Proposition 3).

α	γ	$\mathscr{V}_{\mathrm{tot}}$	L_{model}	L_{data}	total	flips fit	$\tilde{\mathscr{I}}_{\mathrm{flip}}$	$\tilde{\mathscr{I}}_{\mathrm{clean}}$	u
3	6.8	16	47	171	218	1/32	0.078	0.0024	0.31
0.2	66.6	201	249	88	337	10/32	0.744	0.0199	0.32
0.03	118.5	414	442	9	451	31/32	1.585	0.0013	0.51

4.2 Compression sweep: two moons

We trained a 2–32–32–2 tanh MLP ($P=1218$) on 300 two-moons points with 32 flipped labels at $\alpha\in\{3,0.2,0.03\}$ (Figure 1, Table 2). Weakening the prior increases the effective parameters from 6.8 to 118.5 and moves flipped examples from ignored (1/32 fit) to memorized (31/32 fit). The median capacity of a flipped example rises from 0.078 to 1.59 bits, while clean examples stay between 0.001 and 0.02 bits. Total code length grows from 218 to 451 bits: the decrease in data bits (171→9) is outweighed by the increase in model bits (47→442), so the Laplace evidence prefers the most compressed model, the one that ignores the noise. Only 31–51% of the total model information is attributable to single examples.

A side observation: at $\alpha=0.03$ the posterior-averaged fidelity $\bar{F}_i$ reached -41 bits for some examples that θ^* fits well. For confident examples $\Lambda_i\to0$, so the Gaussian's tails extend into regions of very low likelihood. We therefore plot the plug-in fidelity F_i; the capacity measures are unaffected.

4.3 A small vision transformer on digits

We trained a one-block vision transformer (2×2 patches, 16 tokens plus a class token, width 20, two heads; $P=4130$) on 1,000 of scikit-learn's 8×8 digits (Pedregosa et al., 2011) with 63 labels flipped to a random other class, at $\alpha=8$ (Figure 2). The model reached 0.875 test accuracy and fit 58/63 flipped labels, with $\gamma=847$ effective parameters. Flipped examples cost a median of 3.46 bits versus 0.049 bits for clean ones (71×). Of the 16 highest-capacity images, 14 are flips; the other two are atypical but correctly labeled digits (a "2" and an "8"), which is the long-tail behavior predicted by Feldman (2020). Clean examples follow the $\mathscr{I}\approx\mathscr{V}^2$ regime (slope two in Figure 2b), while flips lie above it. The unique fraction is $u=0.38$.

Removing a whole digit class costs 14–25% more volume than the sum of its members' single-example volumes ($\rho=1.14$–1.25), whereas the flipped set ($\rho=1.07$) and a random clean set of the same size ($\rho=1.02$) are nearly additive (Table 3). Members of a class redundantly encode shared structure; flipped labels and random examples do not. The joint $\mathscr{I}$ shows the same ordering more strongly (class ratios 1.8–3.3 versus 1.1 for flips), but it relies on a large joint Newton step and is less reliable than $\mathscr{V}$, which depends only on curvature.

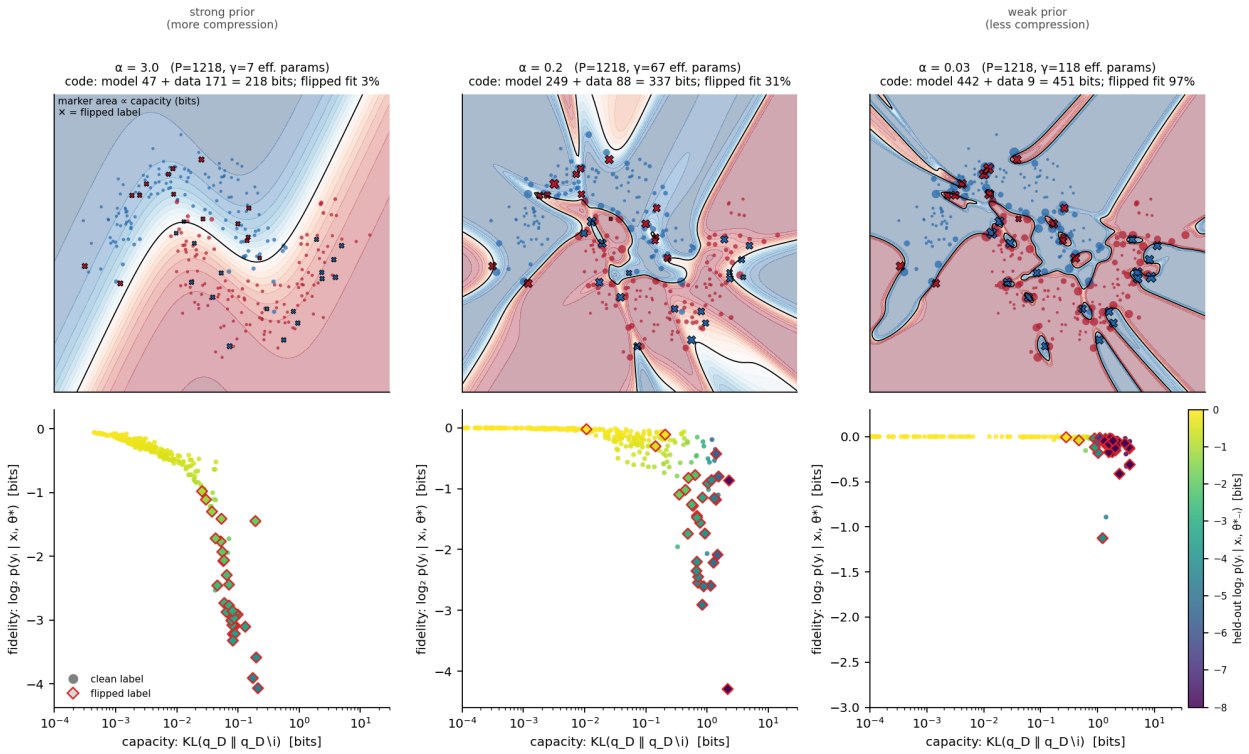


Figure 1: Two moons with 32/300 flipped labels, tanh MLP ($P = 1218$), sweeping the prior precision α. Top: decision function; marker area $\propto \mathscr{I}(\{i\})$; $\times$ marks flipped labels. Bottom: capacity $\mathscr{I}(\{i\})$ versus fidelity F_i; color is the predicted held-out fidelity F_i^-; diamonds with red edges are flipped labels. Titles give the two-part code $L_{\text{model}} + L_{\text{data}}$ in bits. As the prior weakens, flipped examples move from *ignored* (low fidelity, held-out $\approx$ fidelity) to *memorized* (high fidelity, $\sim$1–4 bits each, low held-out fidelity) and carve islands in the decision boundary.

Table 3: Set capacities (bits) for the transformer. "joint" removes the set at once; "sum" adds single-example values. ρ is the volume redundancy ratio.

set	n	$\mathscr{V}$ joint	$\mathscr{V}$ sum	ρ	$\mathscr{I}$ joint	$\mathscr{I}$ sum
class 0	94	71.4	56.9	1.25	62.7	19.2
class 1	95	119.4	97.8	1.22	116.6	46.0
class 2	81	78.2	65.0	1.20	69.7	30.3
class 3	102	113.1	94.5	1.20	93.0	42.5
class 4	91	68.0	57.0	1.19	57.9	22.9
class 5	101	72.2	62.6	1.15	53.2	27.1
class 6	83	88.0	71.5	1.23	66.4	28.4
class 7	102	82.9	67.1	1.24	74.4	28.7
class 8	95	102.3	89.2	1.15	81.7	41.5
class 9	93	84.5	74.4	1.14	57.2	32.4
flipped	63	213.4	200.2	1.07	267.9	242.1
random	63	44.1	43.1	1.02	17.2	15.9

4.4 Validation against retraining

For 40 MLP examples ($\alpha = 0.2$; the 20 highest-capacity and 20 random) and 28 transformer examples (12 highest and 16 random), we retrained without each example by warm-started L-BFGS and measured actual memorization $F_i - \log_2 p(y_i \mid x_i, \theta_{-i}^{\text{retrain}})$ (Figure 3). Capacity ranks actual memorization well: Spearman correlation between $\mathscr{I}(\{i\})$ and actual memorization is 0.84 (MLP) and 0.88 (transformer), monotone over four orders of magnitude. The one-step prediction of memorization, $F_i - F_i^-$, is accurate for typical examples but not for heavily memorized ones. For all 12 highest-capacity transformer examples it overestimated (about 17–60 predicted versus 6–13 actual bits), and for the MLP it was scattered on both sides of the diagonal (Pearson 0.60 overall, 0.90 for the transformer, the latter driven largely by the separation between groups). This matches reports that influence approximations degrade for highly influential points (Basu et al., 2021; Bae et al., 2022). Computing all 1,000 transformer capacities took about 11 s, less than one warm-started retrain ($\approx$14 s).

5 Case study: molecular property prediction

Molecular property data sets are small, assembled from assays with limited reproducibility, often pooled across laboratories, and full of activity cliffs, where near-identical structures have very different properties. They are therefore a natural setting for asking which training compounds a model spends capacity on. We applied capfid to dropout MLPs, a standard baseline for these tasks, using the output-space computation of Section 3.5.

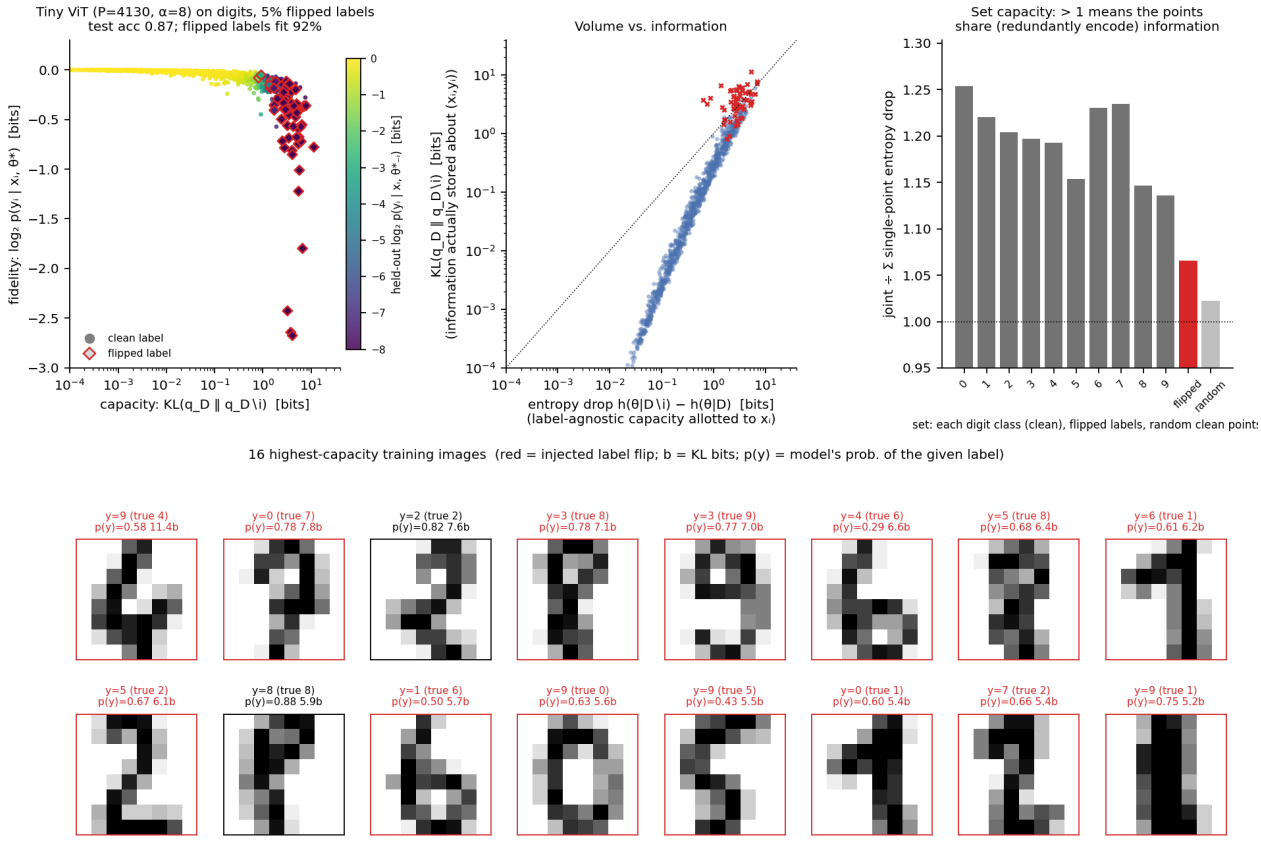


Figure 2: Vision transformer ($P = 4130$, $\alpha = 8$) on 8×8 digits with 63/1000 flipped labels. (a) Capacity versus fidelity, colored by predicted held-out fidelity. (b) Volume $\mathscr{V}$ versus information $\mathscr{I}$ per example: clean examples (blue) follow $\mathscr{I} \approx \mathscr{V}^2$ (in nats); flips (red) store more than their volume. (c) Redundancy ratio $\rho(S)$ for each clean digit class, the flipped set, and a random clean set of equal size. (d) The 16 highest-capacity training images with given and true labels, the model's probability of the given label, and $\mathscr{I}$ in bits; red frames are injected flips.

5.1 Setup

Tasks. Three Therapeutics Data Commons ADMET benchmark tasks (Huang et al., 2021) with the benchmark's scaffold split (seed 1): AMES mutagenicity (Xu et al., 2012), a binary task compiled from several public sources (5,093/728/1,457 train/validation/test compounds); Lipophilicity_AstraZeneca (AstraZeneca, 2016; Wu et al., 2018), octanol/water $\log D_{7.4}$ measured in a single laboratory (2,940/420/840); and Solubility_AqSolDB (Sorkun et al., 2019), aqueous solubility $\log S$ curated from nine public sources (6,986/999/1,995; two test SMILES that RDKit could not parse were dropped).

Features. ECFP4 (Rogers and Hahn, 2010) (Morgan radius 2, 2,048 bits); APDP, atom pairs (Carhart et al., 1985) together with Sheridan's donor/acceptor binding-property pairs (Kearsley et al., 1996; Sheridan, 2019), folded into 2,048 bits; and CheMeleon (Burns et al., 2025), the 2,048-dimensional embedding of a Chemprop message-passing network (Heid et al., 2024) pretrained to predict Mordred descriptors (Moriwaki et al., 2018), standardized on the training set.

Models. ReLU MLPs with dropout (Srivastava et al., 2014) after every hidden layer and one output (a logit, or the mean of standardized targets), trained by minibatch Adam with weight decay. For each task and featurization we drew the same 24 configurations at random (width, depth, dropout, weight decay, learning rate; Appendix D), trained each with early stopping on the validation metric (AUROC for AMES, MAE otherwise), and kept the best. Models have 0.6–4.2 million parameters. We analyze two training regimes with the selected configuration: *early*, stopped at the selected epoch, and *long*, trained for 100 epochs.

Posterior. For each trained network we computed the empirical NTK Gram matrix over the training set and the refit posterior of Section 3.5 with a Bernoulli (AMES) or Gaussian likelihood. The prior precision α maximized the training-set Gaussian-process marginal likelihood (Laplace for AMES). For regression, the noise σ maximized the predictive likelihood of the validation split with α profiled out. Maximizing the training marginal likelihood over σ as well double-dips, because the tangent kernel was fit to the same labels: it chose $\sigma = 0.09$–0.52 for $\log D$ and 0.39–0.93 for $\log S$ (versus 0.61–0.87 and 1.00–1.39 from the validation split), and its test

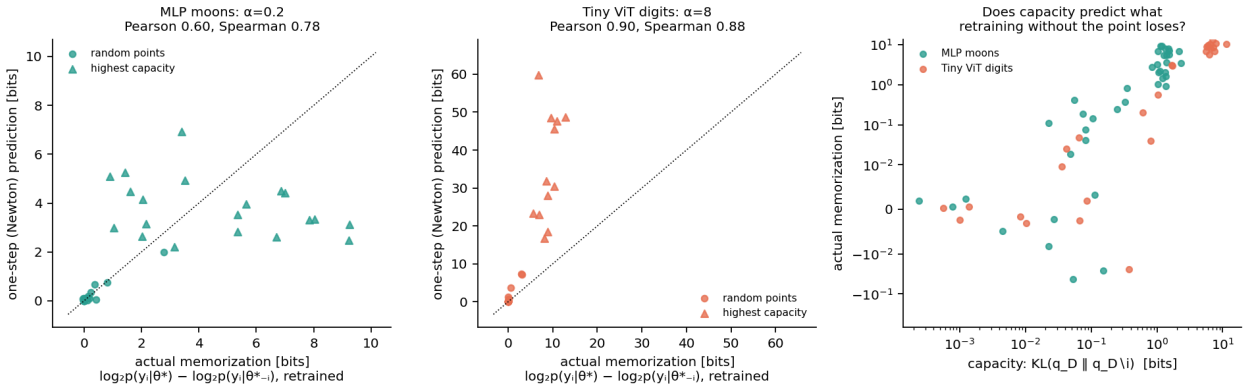


Figure 3: Validation against warm-started retraining. Left, middle: one-step (Newton) prediction of memorization versus actual memorization for the MLP ($\alpha = 0.2$) and the transformer ($\alpha = 8$); triangles are the highest-capacity examples. Right: capacity $\mathscr{I}(\{i\})$ versus actual memorization for both models (symmetric-log y axis).

predictions were overconfident, with standardized test residuals of standard deviation 1.3–3.7 (versus 0.89–1.00). Each posterior, including hyperparameter selection (a median 80% of the time), took 35 s (Lipophilicity) to 7 min (AMES, AqSolDB) on three CPU threads. We analyzed three training seeds of each early-stopped network and two of each long-trained one; per-compound results use seed 0.

Memorization by retraining. To test capacity against the networks themselves rather than their linearization, we trained 48 networks per task, featurization and regime on random 70% subsets of the training set with new seeds, and measured the memorization of each compound as its mean $\log_2$-likelihood under networks that saw it minus networks that did not (Feldman and Zhang, 2020).

Key compounds. To pick compounds without hand selection, we ranked training compounds by the percentile of $\mathscr{I}(\{i\})$ under each featurization's early-stopped model and took (i) the four with the highest mean percentile (*consensus*); (ii) for each featurization, the compound in its top 1% whose percentile most exceeds the larger of the other two (*specific*); and (iii) among compounds with below-median information, the one with the highest mean volume percentile (*atypical*). We depict them with their nearest training neighbor by ECFP4 Tanimoto similarity (Tc) and place them in UMAP embeddings (McInnes et al., 2018) of each feature space.

5.2 Models, posteriors, and validation

The selected networks reached test AUROC 0.76–0.84 on AMES and MAE 0.55–0.67 $\log D$ and 0.82–1.15 $\log S$ (Table 4); CheMeleon features were best on all three tasks. The refit posterior mean, which is what capfid analyzes, matched the networks' test metrics to within 0.016 AUROC and 0.03 log units. Posteriors were stable across training seeds: α varied by at most 25% and γ by at most 7% (coefficient of variation), and per-compound information was rank-correlated between seeds at 0.76–0.95 except for the long-trained AMES networks (0.59–0.86), whose posteriors store almost nothing about most compounds. Rankings were robust to a tenfold weaker prior (Spearman ≥ 0.66 with the selected one) but less so to a tenfold stronger prior (0.32–0.996), which moves compounds from memorized to ignored as in the two-moons sweep.

Per-compound information rank-correlated with memorization measured by retraining at $\rho = 0.53$–0.71 for AMES, 0.38–0.65 for Lipophilicity and 0.49–0.73 for AqSolDB (Figure 4). The one-step estimate from the trained network, which ignores that dropout and early stopping leave $\nabla L(\theta^*) \neq 0$, did better only for the three early-stopped AMES networks (trained for 1–5 epochs) and worse for the other 15 models, with values up to 226 bits for single compounds. The magnitudes differ sharply. Summed over compounds, the networks memorized 4 to 400 times more, measured by retraining, than the refit posterior stores (Table 4). The posterior is a calibrated surrogate whose prior and noise are chosen to predict held-out compounds, whereas the networks interpolate their training labels (train MAE 0.11–0.15 $\log D$ against a validation-selected σ of 0.61–0.87). The posterior also conditions on the network's tangent features, so label information that training has already written into those features is not counted. Training AMES networks to 100 epochs shows this most clearly: memorization by retraining grew 4–13-fold, while the posterior's total information fell 5–20-fold and γ fell by a third or more, because the 100-epoch features separate the training labels on their own (L/N drops to 0.06–0.14 bits). capfid's ranking of compounds is therefore informative across regimes, but its totals compare models only within a training regime.

5.3 What the models spend capacity on

AMES: conflicts with structural alerts. Every one of the 20 compounds with the highest consensus information is mispredicted by all three networks, and 81% of the top 100

Table 4: Case-study models. Test metric: AUROC for AMES (higher is better), MAE in log units otherwise, for the network (mean of three seeds) and for the refit posterior mean (seed 0). $\gamma = d(\mathscr{D})$: effective parameters (Remark 1). L/N: Laplace two-part code length per training compound in bits (continuous for regression, so comparable only within a task). Memorization: totals over training compounds of capfid's information $\mathscr{I}(\{i\})$ and of memorization measured by retraining. Spearman ρ: per-compound memorization by retraining against $\mathscr{I}(\{i\})$ of the refit posterior (refit) or against one-step memorization $F_i - F_i^-$ from θ^* (one step); and $\mathscr{I}(\{i\})$ for two training seeds (seeds).

				test metric				memorization (bits)		Spearman ρ		
Task	Features	Regime	Epochs	network	refit	γ	L/N	capfid	retraining	refit	one step	seeds
AMES	ECFP4	early	1	0.804	0.804	679	0.49	433	688	0.59	0.68	0.94
		long	100	0.803	0.805	284	0.06	22	9,208	0.67	0.42	0.66
	APDP	early	5	0.758	0.774	595	0.54	375	1,033	0.53	0.64	0.93
		long	100	0.716	0.731	381	0.14	90	11,980	0.56	0.51	0.59
	CheMeleon	early	4	0.837	0.833	825	0.31	399	2,106	0.68	0.75	0.95
		long	100	0.820	0.830	542	0.13	86	7,864	0.71	0.59	0.86
Lipo	ECFP4	early	72	0.628	0.620	560	0.97	92	2,529	0.38	0.10	0.78
		long	100	0.623	0.620	566	1.00	96	2,456	0.41	0.10	0.76
	APDP	early	11	0.669	0.681	420	1.38	189	813	0.65	0.61	0.91
		long	100	0.678	0.698	520	1.29	102	1,854	0.44	0.20	0.70
	CheMeleon	early	72	0.551	0.543	369	0.59	41	2,191	0.61	0.31	0.78
		long	100	0.554	0.542	372	0.59	40	2,197	0.60	0.28	0.79
AqSol	ECFP4	early	34	1.145	1.155	1233	0.89	242	4,351	0.51	0.34	0.83
		long	100	1.127	1.156	1265	0.89	271	4,491	0.49	0.27	0.80
	APDP	early	84	1.066	1.070	745	1.04	165	2,537	0.73	0.61	0.85
		long	100	1.075	1.065	780	1.02	177	2,755	0.71	0.60	0.84
	CheMeleon	early	5	0.825	0.801	754	0.64	368	2,487	0.71	0.65	0.95
		long	100	0.885	0.864	780	0.50	146	4,189	0.70	0.52	0.86

have a training neighbor with Tc ≥ 0.5 and the opposite label, against 15% of all training compounds (Figure 5). Most are negatives that carry classical mutagenicity alerts (aryl azides, nitroarenes, aromatic C- and N-nitroso groups, epoxides), and the rest are alert-free positives. The top compound is a diazido-acridine labeled negative whose regioisomer (Tc 0.91) is labeled positive; others are a dinitrophenazine (negative) whose mononitro analogue (Tc 0.88) is positive and prednisone (positive; prednisolone, Tc 0.68, is negative). These are either errors in labels pooled from several sources or real exceptions such as positional effects on metabolic activation; the model cannot tell which, but it isolates a few dozen compounds that an expert can check. Featurization-specific compounds show where a representation, rather than the label, is the problem: compound 5 has no ECFP4 neighbor above Tc 0.26 and costs 1.8 bits under ECFP4 but under 0.01 bits under the other two featurizations.

Lipophilicity: ionization. The highest-capacity compounds are almost all acids, bases, zwitterions or permanent cations, whose $\log D_{7.4}$ departs from intrinsic lipophilicity by an amount set by their pK_a, which none of the representations encodes directly (Figure 6). Several are ionization cliffs: a carboxylic acid at $\log D = -0.87$ whose methyl ester (Tc 0.88) is at 2.72, and 2-phenylquinoline-4-carboxylic acid (-0.70) next to 2-phenylquinolin-4-ol (2.54). Two labels look wrong: 8-hydroxyquinoline-2-carboxylic acid at 3.30 is unexpectedly high for an acid at pH 7.4, and a phosphonooxy amino acid that is multiply charged at pH 7.4 is recorded at 2.50. The same compounds show the capacity–fidelity taxonomy across models. The APDP network (11 epochs, train MAE 0.49) predicts about 1 log unit for all of them and so ignores them, whereas the ECFP4 and CheMeleon networks (72 epochs, train MAE 0.13–0.15) fit them and memorize them, by up to 27 and 59 bits respectively by retraining.

AqSolDB: inorganic compounds and errors. Salts, metal compounds and other inorganics are 14% of the training set but 56–74% of each model's top 1% by information (Figure 7). Two consensus compounds are recorded as nearly insoluble although both are freely water-soluble salts: sodium 4-hydroxybenzenesulfonate ($\log S = -8.36$) and aluminium sulfate hydrate (-7.50).

Representation collisions. Featurization-specific compounds often have an identical twin: n-hexyl acetate (compound 5 in Figure 7) has the same ECFP4 vector as a training compound two log units less soluble. Binary ECFP4 at radius 2 cannot count methylene groups, so homologous series collapse: 16 n-alkanols from C_6 to C_{24} share one vector and span 6 log units, and n-alkanes, fatty acids and their esters behave alike. APDP is empty for compounds whose fragments are single atoms and folds polychlorinated biphenyl congeners together. CheMeleon separates nearly all compounds (Table 5). Per-compound scores understate such groups, because each member is redundant with the rest, so we scored each group jointly with `set_capacity`. The 149 AqSolDB compounds with an all-zero APDP vector, from

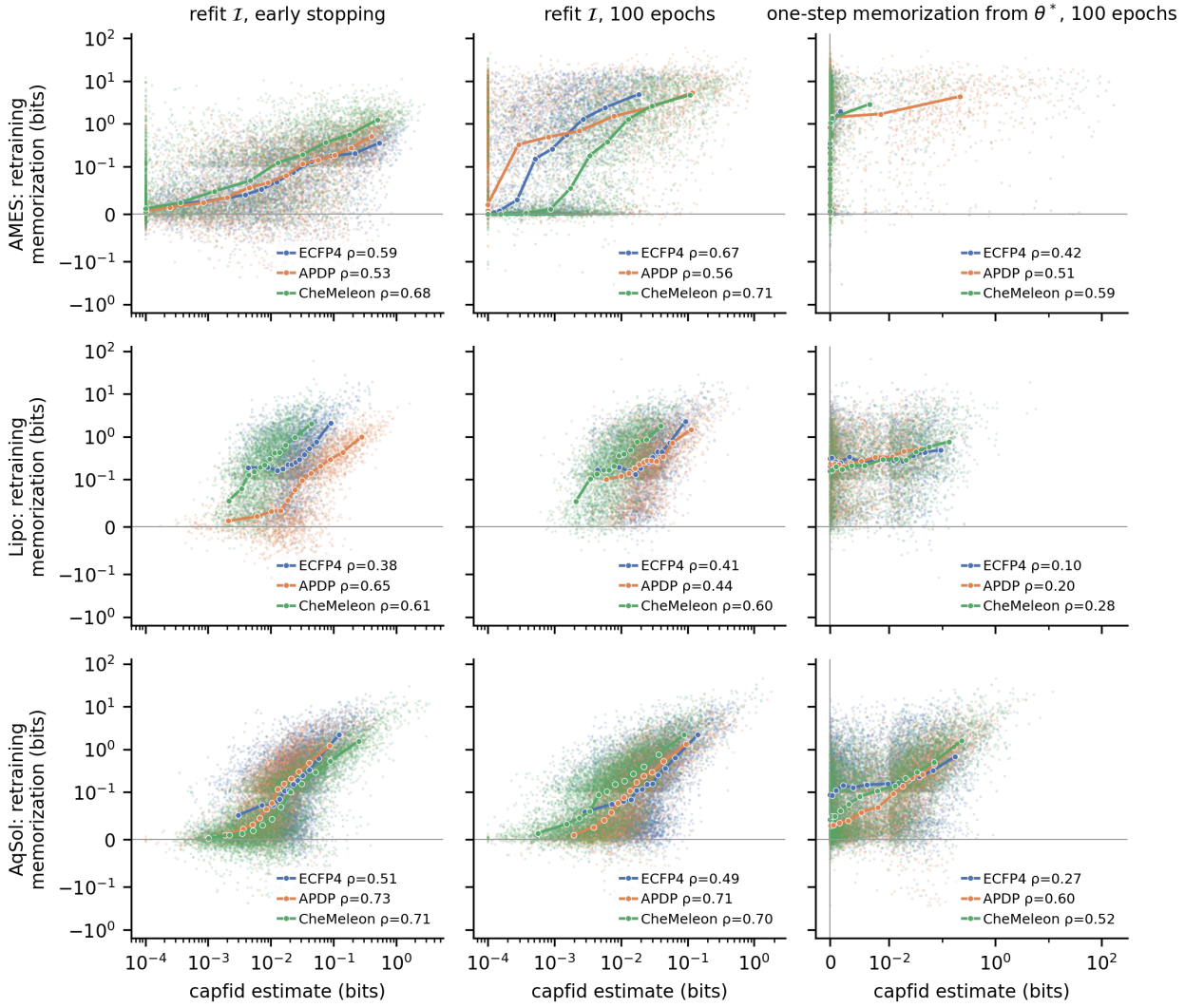


Figure 4: Capacity versus memorization measured by retraining, one point per training compound, for each task (rows) and featurization (colors); lines connect medians of twelve equal-count bins; ρ is Spearman's correlation. Left, middle: information $\mathscr{I}(\{i\})$ of the refit posterior for early-stopped and 100-epoch networks (values below 10^{-4} bits are drawn at 10^{-4}). Right: one-step memorization $F_i - F_i^-$ computed from the trained network θ^* without refitting (symmetric-log axis; it can be negative because $\nabla L(\theta^*) \neq 0$).

chromium metal ($\log S = -10.0$) to ammonium bromide ($+1.9$), jointly determine 0.72 effective parameters and store 7.2 bits, yet cost 669 bits of data code: the model predicts their mean and ignores them as a set. Members of conflicting groups cost 1.4–3.8 times as many bits per compound as other compounds and account for 32% of the APDP model's AqSolDB data code. No amount of capacity can fit these labels; only a better representation can.

5.4 Generalization

Prospective volume flags uncertain test compounds. The prospective volume of a test compound, the capacity it would occupy if added to the training set, requires no label. For AMES it rank-correlated with the networks' absolute test error at $\rho = 0.57$–0.67, and the highest-volume quintile had about three times the error of the lowest (Figure 8). Part of this signal is the predicted probability, since volume grows with the curvature $p(1-p)$. For regression, where volume depends on the input alone, the correlation was weak ($\rho = 0.06$–0.21): most test error in these tasks comes from label noise and cliffs that the posterior regards as aleatoric, not from compounds far from the training set.

Memorization from longer training did not pay. Training AMES networks to 100 epochs instead of the validation-selected 1–5 multiplied their memorization by retraining by 4–13 (Table 4) and drove training AUROC to 1.00, but test AUROC changed by -0.001 (ECFP4), -0.042 (APDP) and -0.017 (CheMeleon). On AqSolDB the CheMeleon network trained to 100 epochs instead of 5 lowered its training MAE from 0.56 to 0.32 and raised its test MAE from 0.83 to 0.89, while memorization by retraining rose from 2,487 to 4,189 bits. Where the selected epoch count was al-

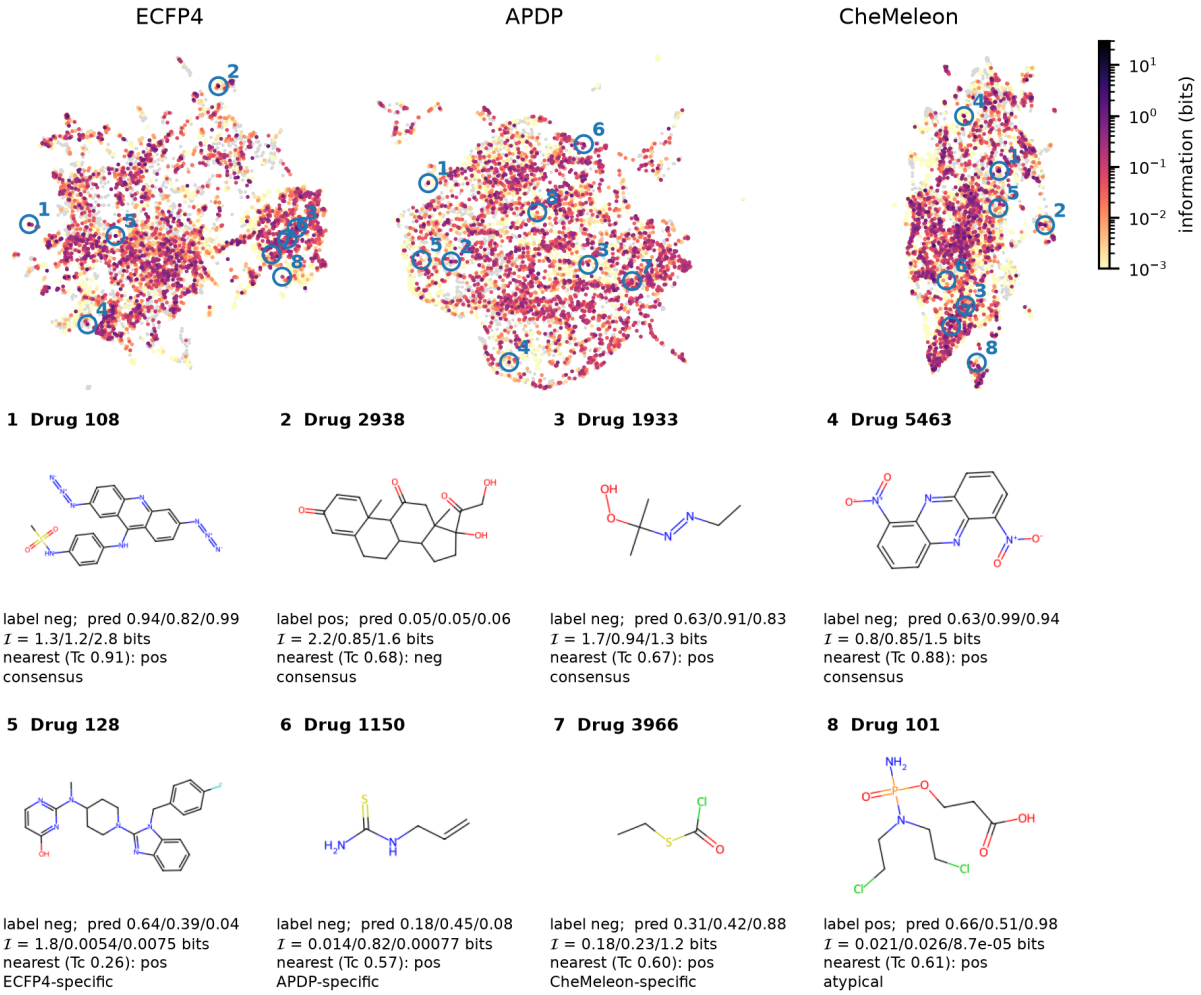
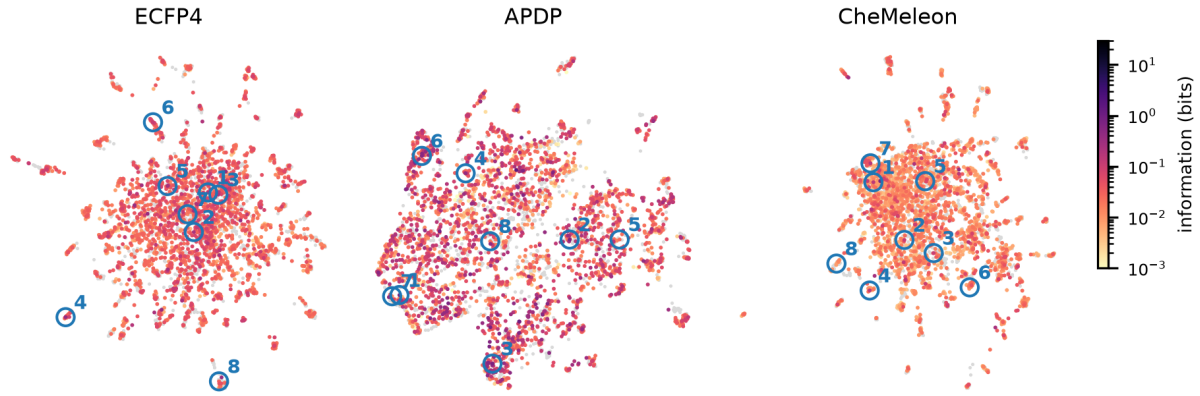


Figure 5: AMES mutagenicity. Top: UMAP embeddings (McInnes et al., 2018) of all compounds in each feature space (Jaccard distance for fingerprints, Euclidean for standardized CheMeleon embeddings). Training compounds are colored by the information $\mathscr{I}(\{i\})$ that the early-stopped model's posterior stores; validation and test compounds are gray. Bottom: key compounds (circled above). Slash-separated values are for ECFP4/APDP/CheMeleon; "pred" is the network's prediction (here, the probability of a positive); "nearest" is the most similar other training compound by ECFP4 Tanimoto similarity, with its label.



1 CHEMBL1368601

label 3.30; pred 3.32/0.94/2.81
$\mathcal{I}$ = 0.2/0.89/0.15 bits
nearest (Tc 0.49): 1.81
consensus

2 CHEMBL1790041

label -0.95; pred -0.64/0.81/-0.17
$\mathcal{I}$ = 0.23/0.72/0.12 bits
nearest (Tc 0.27): 3.30
consensus

3 CHEMBL174107

label -0.84; pred -0.76/1.24/0.03
$\mathcal{I}$ = 0.22/0.8/0.11 bits
nearest (Tc 0.47): 0.79
consensus

4 CHEMBL2062585

label -0.87; pred -0.30/1.08/-0.13
$\mathcal{I}$ = 0.37/0.39/0.14 bits
nearest (Tc 0.88): 2.72
consensus

5 CHEMBL723

label 3.27; pred 3.12/2.91/3.47
$\mathcal{I}$ = 0.26/0.024/0.0097 bits
nearest (Tc 0.44): 0.24
ECFP4-specific

6 CHEMBL1917421

label 3.56; pred 3.59/2.32/3.57
$\mathcal{I}$ = 0.0023/0.54/0.0065 bits
nearest (Tc 0.62): 2.87
APDP-specific

7 CHEMBL139725

label 0.83; pred 0.85/1.04/0.54
$\mathcal{I}$ = 0.012/0.01/0.16 bits
nearest (Tc 0.25): 1.10
CheMeleon-specific

8 CHEMBL272705

label 2.21; pred 2.23/2.69/2.24
$\mathcal{I}$ = 0.03/0.02/0.0065 bits
nearest (Tc 0.63): 1.73
atypical

Figure 6: AstraZeneca lipophilicity ($\log D_{7.4}$); layout as in Figure 5.

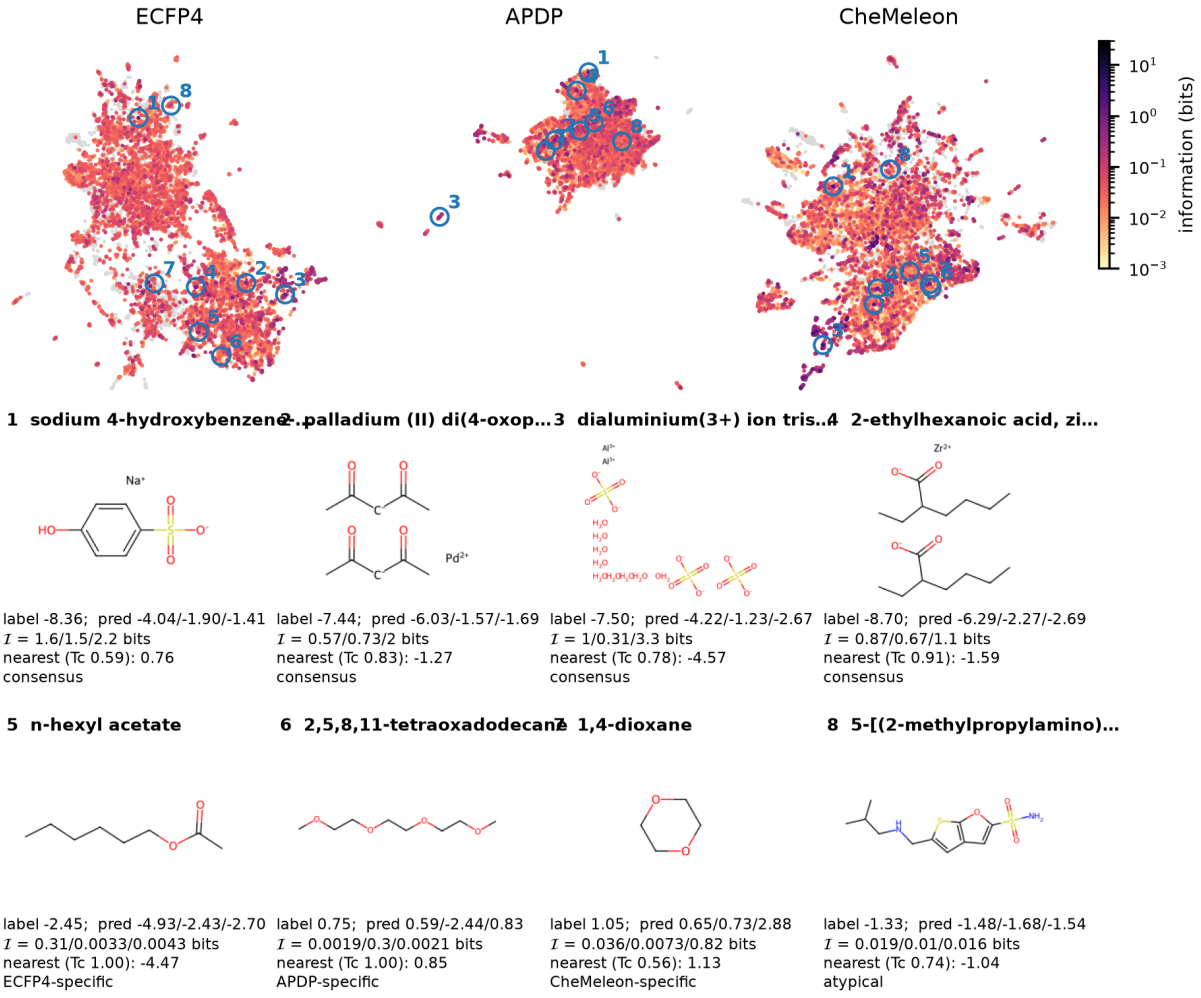


Figure 7: AqSolDB aqueous solubility ($\log S$, mol/L); layout as in Figure 5. Compound 3 (aluminium sulfate hydrate) is drawn apart from the main APDP cloud because its APDP vector is nearly empty.

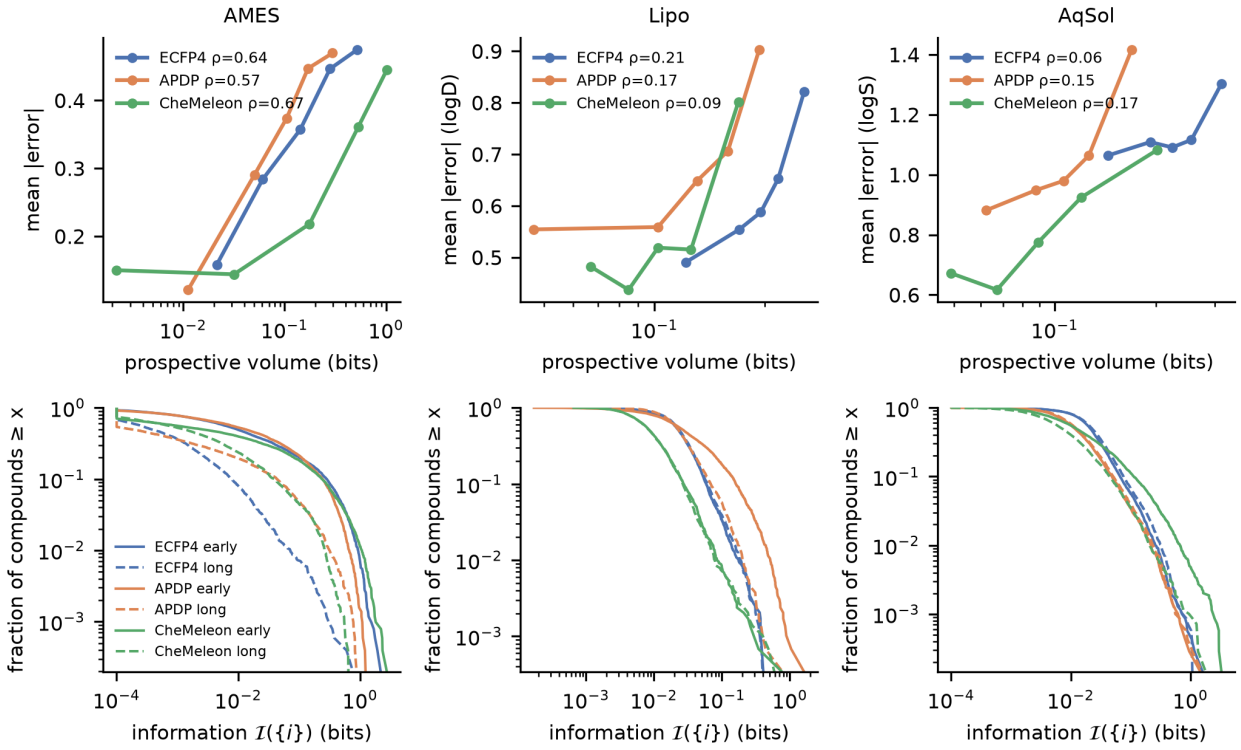


Figure 8: Capacity and generalization. Top: mean absolute test error (probability for AMES) of the early-stopped networks in quintiles of each test compound's prospective volume, which requires no label; ρ is Spearman's correlation over test compounds. Bottom: distribution of the information stored per training compound, early-stopped (solid) versus 100-epoch (dashed) networks.

Table 5: Representation collisions: groups (≥ 2) of training compounds with identical feature vectors. A group conflicts if its labels differ (AMES) or span more than 1 log unit. Code: mean data code length $-\log_2 p(y_i \mid \hat{f}_i)$ per compound (bits) for members of conflicting groups versus all other compounds (early-stopped models).

		all groups		conflicting		
Task	Features	groups	cpds	groups	cpds	code
AMES	ECFP4	365	757	2	4	1.05 / 0.27
	APDP	575	1,345	65	185	1.07 / 0.32
	CheMeleon	17	34	0	0	– / 0.08
Lipo	ECFP4	62	132	2	4	1.33 / 0.66
	APDP	123	277	6	23	1.58 / 1.13
	CheMeleon	0	0	0	0	– / 0.39
AqSol	ECFP4	285	736	81	292	1.18 / 0.57
	APDP	593	1,886	197	1,002	1.90 / 0.68
	CheMeleon	0	0	0	0	– / 0.46

ready near 100 (Lipophilicity ECFP4 and CheMeleon), nothing changed. capfid's per-compound ranking was largely preserved between regimes (Spearman 0.56–0.88), so the compounds memorized by longer training are mostly those that already cost capacity under early stopping.

Description length ranks representations. Within each training regime, the Laplace code length per compound was lowest for CheMeleon on all three tasks, the featurization with the best test metric (Table 4). It also ordered ECFP4 and APDP correctly on AMES and Lipophilicity but not on AqSolDB, where APDP's collisions inflate its code length but its test error is lower.

5.5 Summary

Across three ADMET tasks, capacity located (i) compounds whose labels contradict their nearest neighbors and well-established structural alerts, including likely errors (AMES); (ii) a physical mechanism, ionization, that the representations do not encode (Lipophilicity); (iii) gross label errors and chemical classes outside the representations' scope (AqSolDB); and (iv) collisions where a representation maps compounds with very different labels to the same vector. It ranked compounds by how strongly networks memorize them, as measured by 864 retrainings, from a single posterior per network, and it showed that networks trained past early stopping memorize far more than a calibrated posterior needs without generalizing better.

An interactive version of these per-compound results ships with the code (see Code availability): it colors each dataset's UMAP embeddings by any of the quantities above, in either training regime, depicts a compound with every model's scores and its nearest neighbors' labels when clicked, and draws the members of each collision group.

6 Discussion and limitations

A taxonomy. The capacity–fidelity plane separates (i) *typical* examples (low capacity, high fidelity, held-out $\approx$ fidelity), (ii) *memorized* examples (high capacity, high fidelity, low held-out fidelity), (iii) *ignored* examples (modest capacity, low fidelity), and (iv) *atypical but correct* examples (elevated capacity, high fidelity), the long tail that memorization may legitimately serve (Feldman, 2020). The prior precision moves examples between (iii) and (ii), and the evidence offers a principled stopping point.

Limitations.

- *Locality.* The Laplace posterior describes one basin and ignores the implicit regularization of stochastic training; capacities are defined relative to a prior, as any MDL code requires a precision.
- *One-step unlearning.* Held-out predictions for heavily memorized examples are miscalibrated (up to several-fold); treat capacity as a ranking and held-out fidelity as qualitative in that regime. Joint KL for large sets inherits this problem.
- *Over-dispersion.* Posterior-averaged likelihoods are pessimistic for confident networks at weak priors; this does not affect $\mathscr{V}$, $\mathscr{I}$, or d.
- *Choosing the prior.* Capacities are relative to α (and σ). The case study selects them from data, but the Gaussian-process marginal likelihood of a trained network's tangent kernel is optimistic, since the kernel was fit to the same labels; we therefore used held-out data for σ (Section 5.1). Rankings were stable under a tenfold weaker prior but less so under a tenfold stronger one.
- *Refitting changes what is measured.* For networks away from a stationary point, the refit posterior describes the linearized network's own MAP rather than θ^*; its totals are calibrated but much smaller than the memorization the networks actually exhibit, and the two agree only in ranking.
- *Scale and scope.* The output-space computation costs $O((NC)^3)$, so it covers wide networks with up to a few thousand training examples regardless of P; the parameter-space route is limited to $P \lesssim 10^4$. Neither reaches large training sets with many outputs. Kronecker-factored curvature has made influence functions tractable for language models with tens of billions of parameters (Grosse et al., 2023), but whether capacity estimates remain reliable at that scale is untested.

Unique versus shared information. Leave-one-out quantities measure only *unique* information; Proposition 3 shows they undercount the total, by about half to two-thirds here. A Shapley-style average over removal orders would distribute $\mathscr{V}_{\text{tot}}$ exactly and is a natural extension. Tracing each example's marginal volume across nested subsets of increasing size would give an information-theoretic counterpart of individualized data scaling laws (Covert et al., 2024). It would also test a hypothesis our class-level results suggest but do not establish: that examples in redundant groups (high ρ) are those whose value decays fastest with dataset size, while flipped and atypical examples retain unique information as the dataset grows.

7 Conclusion

Defining per-example capacity by Laplace unlearning yields three interpretable quantities (degrees of freedom, volume, and information) that are exact in linear-Gaussian models, cheap to compute for every example, tied to the generalization gap and to the MDL code length, and empirically predictive of which examples a network has memorized. Plotted against fidelity, they give a direct, per-example view of where a model spends its capacity. Computed in output space, they apply to networks with millions of parameters, where, in the molecular property-prediction case study, they recovered contradictory labels, a missing physical mechanism, and representation collisions without being told what to look for.

Capacity is the cost side of a per-example ledger. Value-of-information analyses in economics (Abadie et al., 2023) and data-valuation and scaling analyses in machine learning (Ghorbani and Zou, 2019; Covert et al., 2024) measure the benefit side: the improvement in downstream decisions or held-out performance that an example buys. The two coincide under a logarithmic utility, where volume is the expected value of an example's label, but not in general. In our experiments, memorizing flipped labels consumed capacity and lengthened the total description of the data, a machine-learning analogue of evidence with negative net value. Measuring costs and benefits on a common scale in bits, and tracking how both scale with dataset size, is a natural next step.

Code availability

The method is implemented in `capfid`, an MIT-licensed PyTorch package available at `https://github.com/maomlab/capfid`. The repository includes unit tests that check Propositions 1 to 3 and Corollary 1 against brute-force refits and the output-space computation against the parameter-space one, the script `examples/paper/reproduce_paper.py`, which regenerates Tables 1 to 3 and Figures 1 to 3 and 9 to 11, and `examples/tdc_casestudy/`, which regenerates the case study (Tables 4 and 5, Figures 4 to 8). The case study's per-compound results are also published as a self-contained interactive dashboard at `https://maomlab.github.io/capfid/examples/tdc_casestudy/dashboard.html`, rebuilt from the same cached tables by `examples/tdc_casestudy/dashboard.py`.

References

A. Abadie, A. Agarwal, G. Imbens, S. Jia, J. McQueen, S. Stepaniants, and S. Torres. Estimating the value of evidence-based decision making. *arXiv:2306.13681*, 2023.

A. Achille, G. Paolini, and S. Soatto. Where is the information in a deep neural network? *arXiv:1905.12213*, 2019.

AstraZeneca. Experimental in vitro DMPK and physicochemical data on a set of publicly disclosed compounds. ChEMBL document CHEMBL3301361, 2016.

J. Bae, N. Ng, A. Lo, M. Ghassemi, and R. Grosse. If influence functions are the answer, then what is the question? In *NeurIPS*, 2022.

S. Basu, P. Pope, and S. Feizi. Influence functions in deep learning are fragile. In *ICLR*, 2021.

J. M. Bernardo. Expected information as expected utility. *The Annals of Statistics*, 7(3):686–690, 1979.

D. Blackwell. Comparison of experiments. In *Proceedings of the Second Berkeley Symposium on Mathematical Statistics and Probability*, pages 93–102, 1951.

L. Blier and Y. Ollivier. The description length of deep learning models. In *NeurIPS*, 2018.

Y. Bu, S. Zou, and V. V. Veeravalli. Tightening mutual information-based bounds on generalization error. *IEEE Journal on Selected Areas in Information Theory*, 1(1):121–130, 2020.

J. W. Burns, A. S. Zalte, C. R. A. Abreu, J. Sieg, C. Feldmann, M. Mathea, and W. H. Green. Deep learning foundation models from classical molecular descriptors. *arXiv:2506.15792*, 2025.

R. E. Carhart, D. H. Smith, and R. Venkataraghavan. Atom pairs as molecular features in structure–activity studies: definition and applications. *J. Chem. Inf. Comput. Sci.*, 25(2):64–73, 1985.

R. D. Cook. Detection of influential observation in linear regression. *Technometrics*, 19(1):15–18, 1977.

I. Covert, W. Ji, T. Hashimoto, and J. Zou. Scaling laws for the value of individual data points in machine learning. In *ICML*, PMLR 235:9380–9406, 2024.

E. Daxberger, A. Kristiadi, A. Immer, R. Eschenhagen, M. Bauer, and P. Hennig. Laplace redux: Effortless Bayesian deep learning. In *NeurIPS*, 2021.

V. Feldman. Does learning require memorization? A short tale about a long tail. In *STOC*, 2020.

V. Feldman and C. Zhang. What neural networks memorize and why: Discovering the long tail via influence estimation. In *NeurIPS*, 2020.

A. Ghorbani and J. Zou. Data Shapley: Equitable valuation of data for machine learning. In *ICML*, 2019.

R. Grosse, J. Bae, C. Anil, N. Elhage, A. Tamkin, A. Tajdini, B. Steiner, D. Li, E. Durmus, E. Perez, E. Hubinger, K. Lukošiūtė, K. Nguyen, N. Joseph, S. McCandlish, J. Kaplan, and S. R. Bowman. Studying large language model generalization with influence functions. *arXiv:2308.03296*, 2023.

P. D. Grünwald. *The Minimum Description Length Principle*. MIT Press, 2007.

H. Harutyunyan, A. Achille, G. Paolini, O. Majumder, A. Ravichandran, R. Bhotika, and S. Soatto. Estimating informativeness of samples with smooth unique information. In *ICLR*, 2021.

E. Heid, K. P. Greenman, Y. Chung, S.-C. Li, D. E. Graff, F. H. Vermeire, H. Wu, W. H. Green, and C. J. McGill. Chemprop: A machine learning package for chemical property prediction. *J. Chem. Inf. Model.*, 64(1):9–17, 2024.

G. E. Hinton and D. van Camp. Keeping the neural networks simple by minimizing the description length of the weights. In *COLT*, 1993.

D. C. Hoaglin and R. E. Welsch. The hat matrix in regression and ANOVA. *The American Statistician*, 32(1):17–22, 1978.

R. A. Howard. Information value theory. *IEEE Transactions on Systems Science and Cybernetics*, 2(1):22–26, 1966.

K. Huang, T. Fu, W. Gao, Y. Zhao, Y. Roohani, J. Leskovec, C. W. Coley, C. Xiao, J. Sun, and M. Zitnik. Therapeutics Data Commons: Machine learning datasets and tasks for drug discovery and development. In *NeurIPS Datasets and Benchmarks*, 2021.

A. Immer, M. Korzepa, and M. Bauer. Improving predictions of Bayesian neural nets via local linearization. In *AISTATS*, 2021.

A. Jacot, F. Gabriel, and C. Hongler. Neural tangent kernel: Convergence and generalization in neural networks. In *NeurIPS*, 2018.

S. K. Kearsley, S. Sallamack, E. M. Fluder, J. D. Andose, R. T. Mosley, and R. P. Sheridan. Chemical similarity using physiochemical property descriptors. *J. Chem. Inf. Comput. Sci.*, 36(1):118–127, 1996.

M. E. Khan, A. Immer, E. Abedi, and M. Korzepa. Approximate inference turns deep networks into Gaussian processes. In *NeurIPS*, 2019.

P. W. Koh and P. Liang. Understanding black-box predictions via influence functions. In *ICML*, 2017.

A. Krause, A. Singh, and C. Guestrin. Near-optimal sensor placements in Gaussian processes: Theory, efficient algorithms and empirical studies. *JMLR*, 9:235–284, 2008.

D. V. Lindley. On a measure of the information provided by an experiment. *The Annals of Mathematical Statistics*, 27(4):986–1005, 1956.

S. M. Lundberg and S.-I. Lee. A unified approach to interpreting model predictions. In *NeurIPS*, 2017.

D. J. C. MacKay. A practical Bayesian framework for backpropagation networks. *Neural Computation*, 4(3):448–472, 1992.

J. Martens and R. Grosse. Optimizing neural networks with Kronecker-factored approximate curvature. In *ICML*, 2015.

L. McInnes, J. Healy, and J. Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv:1802.03426*, 2018.

H. Moriwaki, Y.-S. Tian, N. Kawashita, and T. Takagi. Mordred: a molecular descriptor calculator. *J. Cheminform.*, 10:4, 2018.

F. Pedregosa et al. Scikit-learn: Machine learning in Python. *JMLR*, 12:2825–2830, 2011.

G. Pruthi, F. Liu, S. Kale, and M. Sundararajan. Estimating training data influence by tracing gradient descent. In *NeurIPS*, 2020.

H. Raiffa and R. Schlaifer. *Applied Statistical Decision Theory*. Division of Research, Harvard Business School, 1961.

C. E. Rasmussen and C. K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006.

M. T. Ribeiro, S. Singh, and C. Guestrin. "Why should I trust you?": Explaining the predictions of any classifier. In *KDD*, 2016.

J. Rissanen. Modeling by shortest data description. *Automatica*, 14(5):465–471, 1978.

H. Ritter, A. Botev, and D. Barber. A scalable Laplace approximation for neural networks. In *ICLR*, 2018.

D. Rogers and M. Hahn. Extended-connectivity fingerprints. *J. Chem. Inf. Model.*, 50(5):742–754, 2010.

R. P. Sheridan. Interpretation of QSAR models by coloring atoms according to changes in predicted activity: How robust is it? *J. Chem. Inf. Model.*, 59(4):1324–1337, 2019.

M. C. Sorkun, A. Khetan, and S. Er. AqSolDB, a curated reference

set of aqueous solubility and 2D descriptors for a diverse set of compounds. *Sci. Data*, 6:143, 2019.

N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *JMLR*, 15:1929–1958, 2014.

N. Tishby, F. C. Pereira, and W. Bialek. The information bottleneck method. *arXiv:physics/0004057*, 2000.

Z. Wu, B. Ramsundar, E. N. Feinberg, J. Gomes, C. Geniesse, A. S. Pappu, K. Leswing, and V. Pande. MoleculeNet: a benchmark for molecular machine learning. *Chem. Sci.*, 9(2):513–530, 2018.

C. Xu, F. Cheng, L. Chen, Z. Du, W. Li, G. Liu, P. W. Lee, and Y. Tang. In silico prediction of chemical Ames mutagenicity. *J. Chem. Inf. Model.*, 52(11):2840–2847, 2012.

A. Xu and M. Raginsky. Information-theoretic analysis of generalization capability of learning algorithms. In *NeurIPS*, 2017.

C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals. Understanding deep learning requires rethinking generalization. In *ICLR*, 2017.

A A worked example: capacity in Bayesian linear regression

This appendix works Definition 1 out in the simplest setting that contains all of its ingredients: linear regression with a Gaussian prior and Gaussian noise. There the Laplace posterior of Equation (2) is not an approximation, each of the three capacities collapses to a formula in two scalars per example, and each of those formulas turns out to be a familiar object from classical regression diagnostics. The appendix is self-contained, and ends with the experiment of Section 4.1.

A.1 A posterior is a cloud of plausible weights

Write the model as $f(x;w) = \phi(x)^\top w$, where $\phi : \mathscr{X} \to \mathbb{R}^P$ is a fixed feature map — polynomials, random Fourier features, or the last hidden layer of a trained network. Observations are $y_i = f(x_i;w) + \varepsilon_i$ with independent noise $\varepsilon_i \sim \mathcal{N}(0,\sigma^2)$, and the prior is $w \sim \mathcal{N}(0,\alpha^{-1}I)$: before seeing data we believe each weight is of size roughly $\alpha^{-1/2}$. Stacking the $\phi(x_i)^\top$ into $\Phi \in \mathbb{R}^{N\times P}$, Bayes' rule gives a Gaussian posterior

$$q_{\mathscr{D}} = \mathcal{N}(\hat{w},\Sigma),\ \ \Sigma^{-1} = A = \tfrac{1}{\sigma^2}\Phi^\top\Phi + \alpha I,\ \ \hat{w} = \tfrac{1}{\sigma^2}\Sigma\Phi^\top y. \tag{7}$$

Here $\hat{w}$ is ridge regression with penalty $\alpha\sigma^2$, and A is the *precision*: large in directions the data pin down. This is exactly Equation (2) with $J_i = \phi(x_i)^\top$ and $\Lambda_i = \sigma^{-2}$, and nothing has been approximated, because the objective Equation (1) is exactly quadratic in w.

Picture $q_{\mathscr{D}}$ as an ellipsoidal cloud of the weight vectors that remain plausible (Figure 9b). The prior is a round cloud of radius $\alpha^{-1/2}$. Every observation squeezes it, and squeezes it along one direction only, namely $\phi(x_i)$. That is the entire mechanism: the three capacities are three ways of measuring one squeeze.

A.2 Unlearning one observation, and leverage

To unlearn example i we subtract its contribution to the precision, $A_{-i} = A - \sigma^{-2}\phi_i\phi_i^\top$ (Equation (3)), and recentre the cloud on the ridge estimate of the remaining $N-1$ points. The cloud grows, because less data constrains w less, and it moves, because the remaining points prefer a slightly different w (Figure 9b).

A single number governs the growth. Define the *leverage*

$$h_i = \frac{\phi_i^\top\Sigma\,\phi_i}{\sigma^2} \in [0,1), \tag{8}$$

the posterior variance of the model's prediction at x_i, in units of the noise variance; it is the scalar $K_{\{i\}}$ of Proposition 1. It is also the ith diagonal entry of the ridge hat matrix $H = \Phi(\Phi^\top\Phi + \alpha\sigma^2 I)^{-1}\Phi^\top$, which maps labels to fitted values, $\hat{f} = Hy$. Hence

$$h_i = \frac{\partial \hat{f}_i}{\partial y_i}: \tag{9}$$

leverage is the fraction of its own label that the fitted value echoes back. When $h_i \approx 0$ the prediction at x_i is dictated by the other examples; when $h_i \to 1$ the model simply repeats y_i. Leverage is large where the data are sparse — the ends of the x range in Figure 11b — and where $\phi(x_i)$ points somewhere no other example does.

Two classical identities fall out of the same algebra, both exact here:

$$y_i - f(x_i;\hat{w}_{-i}) = \frac{e_i}{1-h_i}, \qquad e_i = y_i - \hat{f}_i, \tag{10}$$

$$\textstyle\sum_i h_i = \operatorname{tr} H = \gamma. \tag{11}$$

Equation (10) is the PRESS formula: what the model would have predicted had it never seen example i costs nothing beyond knowing h_i. Equation (11) is the additivity of Remark 1 in this special case: the leverages are exact shares of a budget, the effective number of parameters $\gamma = d(\mathscr{D})$, which runs from 0 under a strong prior to $\min(N,P)$ under a flat one.

A.3 The three capacities

Putting $C = 1$, $J_i = \phi_i^\top$, $\Lambda_i = \sigma^{-2}$ and $r_i = -e_i/\sigma^2$ into Proposition 1 gives, for a single example,

$$d(\{i\}) = h_i, \tag{12}$$

$$\mathscr{V}(\{i\}) = -\tfrac{1}{2}\log_2(1-h_i), \tag{13}$$

$$\mathscr{I}(\{i\}) = \frac{1}{2\ln 2}\Big[\underbrace{\frac{e_i^2}{\sigma^2}\,\frac{h_i}{1-h_i}}_{\text{surprise}}\underbrace{-h_i - \log(1-h_i)}_{\text{shape}}\Big]. \tag{14}$$

Everything an example costs this model is therefore a function of just two numbers: where it sits, through h_i, and how wrong the fit is there, through e_i (Figure 10).

Degrees of freedom is the leverage itself, measured in parameters rather than bits. By Equation (11) it is example i's exact share of the γ parameters the data determine.

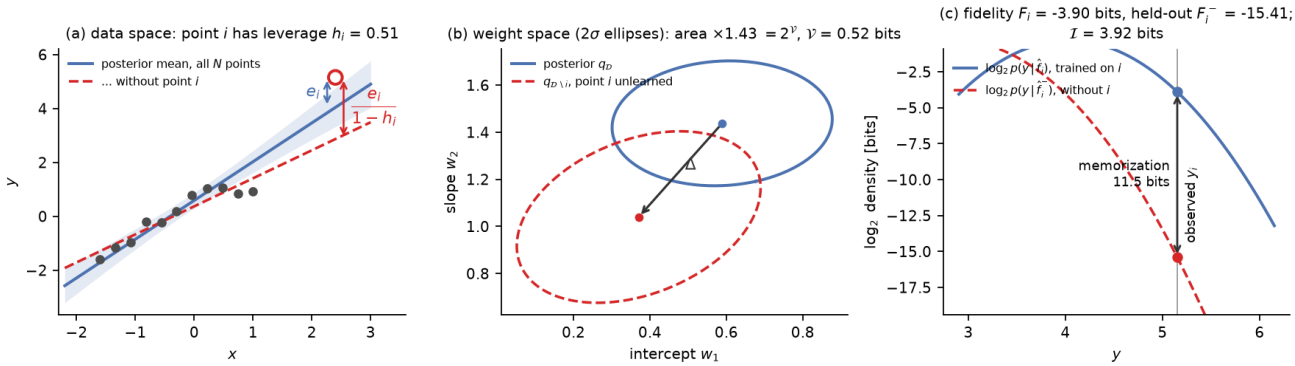


Figure 9: What unlearning one observation does. A straight-line fit ($P = 2$, so the posterior can be drawn) to twelve points, one of which sits apart from the rest and disagrees with them. **(a)** Data space: the posterior mean with and without that point. Its residual e_i is what the fit still misses after absorbing it; $e_i/(1-h_i)$ is what the fit misses when it has not seen it. **(b)** Weight space: unlearning the point inflates the posterior ellipse by $(1-h_i)^{-1/2}$ in area — that factor, in bits, is $\mathcal{V}(\{i\})$ — and shifts its centre by Δ, which is what $\mathcal{I}(\{i\})$ adds on top of $\mathcal{V}$. **(c)** The label's own likelihood on a $\log_2$ axis, so that the two marked heights are the fidelity F_i and the held-out fidelity F_i^- in bits and the gap between them is the memorization. Here one point is worth half a bit of volume but costs the model 11.5 bits of fit when it is taken away.

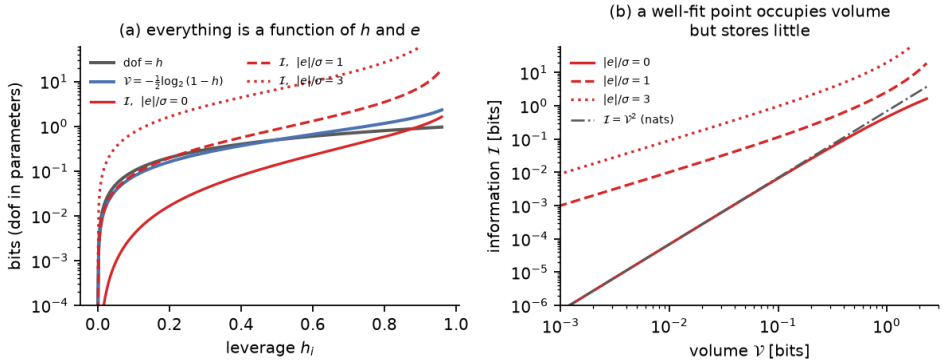


Figure 10: The three capacities as functions of leverage and residual, from Equations (12) to (14) alone; no data are involved. **(a)** All three grow with leverage and diverge as $h \to 1$, where the model is reduced to repeating the label. d and $\mathcal{V}$ agree to first order; they differ only once h is large. **(b)** The same curves in the capacity plane. A perfectly fit example ($e_i = 0$) lies on $\mathcal{I} = \mathcal{V}^2$ in nats; the residual moves it vertically, by a factor of about e_i^2/σ^2 in the surprise term.

Volume is measured in bits. Unlearning example i is a rank-one change to the precision, so the cloud grows in exactly one direction — by the factor $(1-h_i)^{-1/2}$ — and is unchanged in the $P-1$ directions conjugate to ϕ_i. The ellipsoid's volume therefore grows by that same factor, and $\mathcal{V}(\{i\})$ is its base-2 logarithm (Figure 9b). One bit of volume means the example halves the set of weight vectors that remain plausible. Note what is *not* in Equation (13): the label. Volume is settled by where an example sits in feature space, before anyone reads y_i. Corollary 1 says what it then is — the information the model would gain from *any* plausible label at that location, averaged over them — which is why we call it the capacity allotted to an input rather than the capacity used.

Information is the one that reads the label, through the surprise term, which is the squared length of the mean shift Δ of Figure 9b in the metric of the unlearned posterior. The shape term is what is left when the label is exactly as predicted, and for small h it is $h^2/(4\ln 2)$ while $\mathcal{V}$ is $h/(2\ln 2)$: a well-fit example has $\mathcal{I} \approx \mathcal{V}^2$ in nats. That square is the floor in Figure 10b and Figure 11d, the locus of examples that occupy volume but store nothing, and a residual of a couple of σ lifts an example orders of magnitude above it.

A.4 Fidelity, held-out fidelity, and memorization

Fidelity is just the log-likelihood of the label under the fitted model, $F_i = -e_i^2/(2\sigma^2 \ln 2) - \frac{1}{2}\log_2(2\pi\sigma^2)$, and by Equation (10) the held-out fidelity F_i^- is the same expression with e_i replaced by $e_i/(1-h_i)$. Their difference,

$$F_i - F_i^- = \frac{e_i^2}{2\sigma^2 \ln 2}\left[\frac{1}{(1-h_i)^2} - 1\right], \tag{15}$$

is the memorization: the bits of its own label an example loses when the model has not seen it. It needs *both* ingredients. A badly fit example in a crowded region (e_i large, $h_i \approx 0$) is simply misfit, and removing it changes nothing; a well-fit

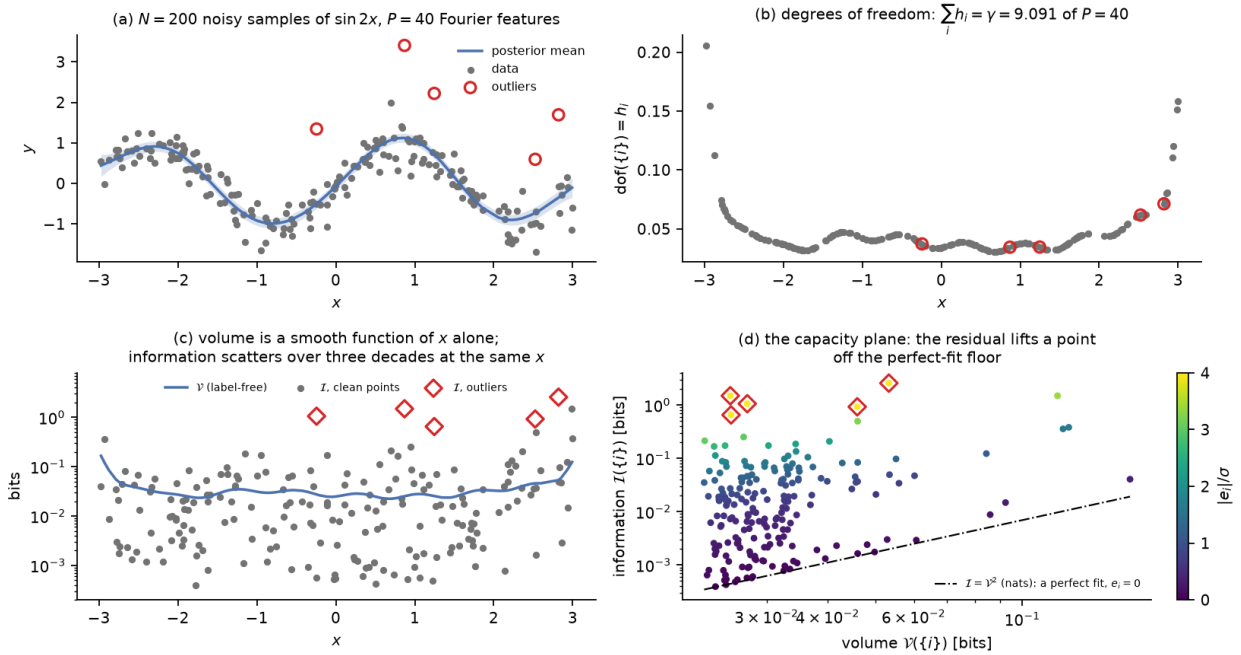


Figure 11: The three capacities across the experiment of Section 4.1: $N = 200$ noisy samples of $\sin 2x$ fitted with $P = 40$ random Fourier features, five of whose targets are shifted by $+2$ (red). **(a)** The fit, with a 2σ posterior band. **(b)** Degrees of freedom $d(\{i\}) = h_i$, a function of x alone, summing to $\gamma = d(\mathscr{D}) = 9.091$ of the 40 nominal parameters (Remark 1). **(c)** $\mathscr{V}$ (line) and $\mathscr{I}$ (points) against x: volume is label-free and smooth, while information varies over three decades at the same input according to the residual. **(d)** The capacity plane, coloured by $|e_i|/\sigma$. The dash-dotted line $\mathscr{I} = \mathscr{V}^2$ (in nats) is the floor reached only by a perfectly fit example; the outliers sit two decades above it at ordinary volume.

example on its own (h_i large, $e_i \approx 0$) is predicted by the rest of the data anyway. Only an example that is both isolated and surprising is memorized.

A.5 The experiment of Section 4.1

Figure 11 is that experiment: $N = 200$ noisy samples of $\sin 2x$, $P = 40$ random Fourier features, $\sigma = 0.3$, $\alpha = 1$, and five targets shifted by $+2$. Panel (b) plots the leverage. It depends on x alone and is largest at the ends of the range, where there are no neighbours on one side; the five outliers are unremarkable in it, because shifting a label does not move the example in feature space. Panel (c) contrasts $\mathscr{V}$ and $\mathscr{I}$ at the same inputs: $\mathscr{V}$ is a smooth curve through x, while $\mathscr{I}$ scatters over three decades at any given x according to the residual there, and the outliers sit two decades above the rest. Panel (d) is the capacity plane: the bulk lies in a band above the perfect-fit floor set by the noise, and the outliers lie far above it at ordinary volume. The five carry a mean $\mathscr{I}$ of 1.34 bits against 0.044 bits for the rest (Section 4.1).

Computing Equations (12) to (14) from h_i and e_i alone and comparing with what `capfid` returns from the general parameter-space algorithm agrees to 3.2×10^{-13} bits over all 200 examples, and Table 1 compares the same quantities against brute-force refits without each point.

What survives beyond this example. Nothing in Definition 1 assumed a linear model. For a network, Equation (2) replaces the exact posterior by the Gaussian matching the curvature of L at θ^*, the scalar h_i becomes the $C \times C$ matrix $K_{\{i\}} = \Lambda_i^{1/2} J_i \Sigma J_i^\top \Lambda_i^{1/2}$, and Equations (12) to (14) become the trace, log-determinant and quadratic form of Proposition 1. The interpretations above carry over unchanged. What does not carry over is exactness: Equation (10) becomes a single Newton step rather than an identity, which is accurate for typical examples and optimistic for heavily memorized ones (Figure 3) — the main limitation of the method. Section 3.5 describes what to do when the network is not at a mode, so that $\nabla L(\theta^*) \neq 0$ and even the Newton step misattributes the leftover gradient.

B Proofs

Proposition 1. Write $H_S = J_S^\top \Lambda_S J_S$ so $A_{-S} = A - H_S$. (i) $d(S) = \mathrm{tr}(\Sigma H_S) = \mathrm{tr}(\Lambda_S^{1/2} J_S \Sigma J_S^\top \Lambda_S^{1/2}) = \mathrm{tr}\, K_S$ by cyclicity. (ii) With $h(\mathcal{N}(\mu, C)) = \frac{1}{2}\log\det(2\pi e C)$, $\mathscr{V}(S) = \frac{1}{2}[\log\det A - \log\det A_{-S}] = -\frac{1}{2}\log\det(I_P - \Sigma H_S) = -\frac{1}{2}\log\det(I_{mC} - K_S)$ by Sylvester's identity $\det(I_P - UW) = \det(I_{mC} - WU)$ with $U = \Sigma J_S^\top \Lambda_S^{1/2}$, $W = \Lambda_S^{1/2} J_S$. (iii) For Gaussians, $\mathrm{KL}(\mathcal{N}(\mu_0, A^{-1}) \| \mathcal{N}(\mu_1, A_{-S}^{-1})) = \frac{1}{2}[\mathrm{tr}(A_{-S}\Sigma) - P + \Delta^\top A_{-S}\Delta + \log\det A - \log\det A_{-S}]$ with $\Delta = \mu_1 - \mu_0 = A_{-S}^{-1} J_S^\top r_S$. Here $\mathrm{tr}(A_{-S}\Sigma) - P = -\mathrm{tr}(H_S\Sigma) = -\mathrm{tr}\, K_S$ and $\Delta^\top A_{-S}\Delta = r_S^\top J_S A_{-S}^{-1} J_S^\top r_S = r_S^\top S_{-S} r_S$. (iv) Woodbury gives $A_{-S}^{-1} = \Sigma + \Sigma J_S^\top \Lambda_S^{1/2}(I - K_S)^{-1}\Lambda_S^{1/2} J_S \Sigma$; pre- and post-

multiplying by J_S and $J_S^\top$ yields S_{-S}. The linearized held-out logits are $f(x_S) + J_S\Delta = f(x_S) + S_{-S}r_S$. □

Proposition 2. Let $Z_S = \int q_{\mathscr{D}_{\setminus S}}(\theta)p(y_S \mid \theta)\,d\theta = \mathbb{E}_{q_{\mathscr{D}_{\setminus S}}} p(y_S \mid \theta)$. Then $q_{\mathscr{D}} = q_{\mathscr{D}_{\setminus S}} p(y_S \mid \theta)/Z_S$, so $\mathrm{KL}(q_{\mathscr{D}} \| q_{\mathscr{D}_{\setminus S}}) = \mathbb{E}_{q_{\mathscr{D}}}[\log p(y_S \mid \theta)] - \log Z_S$. □

Corollary 1. With a Gaussian likelihood and linear model, $q_{\mathscr{D}}$ has covariance independent of y_S. Averaging the KL over $y_S \sim p(\cdot \mid \mathscr{D}_{\setminus S})$ gives the mutual information $I(\theta; y_S \mid \mathscr{D}_{\setminus S}) = h(q_{\mathscr{D}_{\setminus S}}) - h(q_{\mathscr{D}}) = \mathscr{V}(S)$. For a single point in linear regression, $K_{\{i\}} = \sigma^{-2} x_i^\top \Sigma x_i = h_{ii}$, the diagonal of the Bayesian ridge hat matrix $X(X^\top X + \alpha\sigma^2 I)^{-1}X^\top$. □

Proposition 3. Let $g(T) = \frac{1}{2}\log\det(\alpha I + \sum_{i\in T} H_i)$, which is monotone and submodular in T (Krause et al., 2008). Then $\mathscr{V}(S) = g(\mathscr{D}) - g(\mathscr{D}_{\setminus S})$. Ordering $S = \{s_1, \ldots, s_m\}$ and telescoping, $\mathscr{V}(S) = \sum_k [g(\mathscr{D}_{\setminus S} \cup \{s_{1..k}\}) - g(\mathscr{D}_{\setminus S} \cup \{s_{1..k-1}\})] \geq \sum_k [g(\mathscr{D}) - g(\mathscr{D}_{\setminus s_k})] = \sum_k \mathscr{V}(\{s_k\})$, since $\mathscr{D}_{\setminus S} \cup \{s_{1..k-1}\} \subseteq \mathscr{D}_{\setminus s_k}$. Taking the prior as the base set gives the bound on $\mathscr{V}_{\mathrm{tot}} = g(\mathscr{D}) - g(\emptyset)$. □

Two-part code. The Laplace evidence is $\log p(\mathscr{D}) \approx \log p(\mathscr{D} \mid \theta^*) + \log p(\theta^*) + \frac{P}{2}\log 2\pi - \frac{1}{2}\log\det A = \log p(\mathscr{D} \mid \theta^*) - \frac{\alpha}{2}\|\theta^*\|^2 - \frac{1}{2}\log\det(A/\alpha)$, and $\frac{1}{2}\log\det(A/\alpha) = \mathscr{V}_{\mathrm{tot}}$. Using $\alpha\,\mathrm{tr}\,\Sigma = P - \gamma$ in the Gaussian KL to the prior gives $\mathrm{KL}(q_{\mathscr{D}} \| \mathrm{prior}) = L_{\mathrm{model}} - \gamma/2$.

C Experimental details

Two moons. 300 points (`make_moons`, noise 0.2, seed 0); 32 labels flipped. MLP 2–32–32–2 with tanh ($P = 1218$). Objective Equation (1) over all parameters; Adam (3,000 steps, learning rate 10^{-2}, cosine schedule) followed by L-BFGS (strong Wolfe line search, history 50, up to 1,000 iterations). A preliminary sweep gave 4/32 flips fit at $\alpha = 0.3$ and 27/32 at $\alpha = 0.1$, so the transition to memorization is sharp; we show $\alpha = 0.2$ as an intermediate case. Each α took 4–7 s including training.

Digits transformer. scikit-learn digits (1,797 images, pixels scaled to $[0,1]$), random split 1,000/797; 63 training labels replaced by a uniformly random other class. Architecture: linear patch embedding of 2×2 patches, learned class token and positional embeddings, one pre-LayerNorm block (two heads, width 20, GELU MLP of width 40), final LayerNorm and linear head ($P = 4130$). Adam (3,000 steps, 3×10^{-3}, cosine) then L-BFGS (up to 500 iterations). An initial run with $\alpha = 1$ gave 0.83 test accuracy and fit all 63 flips; we report $\alpha = 8$. Training took 184 s; Jacobians, GGN, Cholesky and all per-example quantities about 11 s; the 12 set capacities about 13 s.

Laplace and fidelity. Exact GGN in float64; Jacobians by `torch.func.vmap(jacrev)`. Posterior expectations used 2,000 (MLP) or 1,000 (transformer) Monte Carlo samples with common random numbers across the with/without posteriors.

Retraining. Warm-started from θ^*, L-BFGS for up to 300 iterations on Equation (1) without the example. Warm starting isolates the local effect that the Laplace approximation targets; retraining from scratch would add basin-to-basin variation.

Hardware and reproduction. One CPU core, 3 GB RAM. The full pipeline including validation (`examples/paper/reproduce_paper.py` in the `capfid` repository) ran in about 12 minutes.

D Case-study details

Data and features. Tasks were loaded with PyTDC 0.4.1 (`admet_group`, `get_train_valid_split(seed=1, split_type="default")`). ECFP4 used RDKit's Morgan generator (radius 2, 2,048 bits). APDP followed the implementation in `maomlab/MPLearn`: the nonzero keys of RDKit's atom-pair fingerprint and of Sheridan's binding-property-pair fingerprint (the latter offset by 2^{23}) were folded modulo 2,048 into one binary vector. Because 2^{23} is a multiple of 2,048, both families share the same bits. Atom pairs and binding-property pairs are defined only within a connected fragment, so 149 AqSolDB compounds consisting only of monatomic ions or single atoms have an all-zero APDP vector. CheMeleon embeddings were computed with the published message-passing weights (Zenodo record 15460715) and mean aggregation, as in `chemeleon_fingerprint.py`.

Training. Minibatches of 128, Adam with coupled weight decay, binary cross-entropy or $\frac{1}{2}$MSE on standardized targets. The 24 configurations were drawn without replacement from width $\in \{256, 512, 1024\}$, depth $\in \{1, 2, 3\}$ hidden layers, dropout $\in \{0.1, 0.2, 0.3, 0.5\}$, weight decay $\in \{10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}\}$ and learning rate $\in \{10^{-4}, 3 \times 10^{-4}, 10^{-3}\}$, each trained for up to 200 epochs with patience 25 on the validation metric. Final models were retrained with the selected configuration for the selected number of epochs (early) or 100 epochs (long), with seeds 0, 1 and 2; test metrics are means over seeds and capacity analyses use seed 0.

Posterior. The NTK Gram matrices over the training set and between held-out and training compounds were assembled from layer factors (Section 3.5) in float64. The prior precision maximized the training-set marginal likelihood of the Gaussian process with kernel G/α and mean m: in closed form from one eigendecomposition of G for Gaussian likelihoods (Rasmussen and Williams, 2006, eq. 2.30), and by the

Laplace approximation for AMES (Rasmussen and Williams, 2006, eq. 3.32), with Newton mode-finding at each candidate; both by bounded scalar search over $\log \alpha$. For regression, σ maximized the closed-form predictive log-likelihood of the validation compounds (Rasmussen and Williams, 2006, eqs. 2.25–2.26), with α re-optimized at each candidate σ. For comparison we also maximized the training marginal likelihood jointly over (α, σ) (Nelder–Mead from four starting points). The prior mean m was 0 for standardized targets and the log-odds of the training positive rate for AMES. Posterior expectations used 1,000 Monte Carlo samples for AMES (closed form for Gaussian likelihoods).

Retraining. For model $k = 1, \ldots, 48$, each training compound was kept independently with probability 0.7 and the network was trained from a new initialization (seed $1000 + k$) with the same configuration and number of epochs. Each compound was held out of about 14 networks.

Compute. All case-study stages ran on a 12-core Apple M2 Max CPU: hyperparameter search (216 runs) about 15 minutes on 10 processes; capacity analysis of 45 networks 143 minutes of CPU time in total (35 s to 7 min per network), run as 4 concurrent processes of 3 threads each; 864 retraining runs about 100 minutes on 8 processes. The script `examples/tdc_casestudy/run_casestudy.py` reproduces every number.